

Paralelni sistemi

Rešeni blanketi sa usmenog dela ispita
Akreditacija 2013

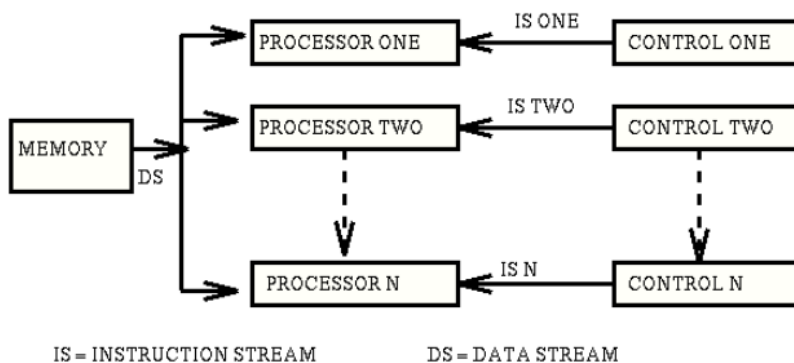
Uvod

1. Navesti Flynn-ovu klasifikaciju računarskih arhitektura. Ukratko objasniti karakteristike svake klase i nacrtati konceptijski dijagram arhitekture.

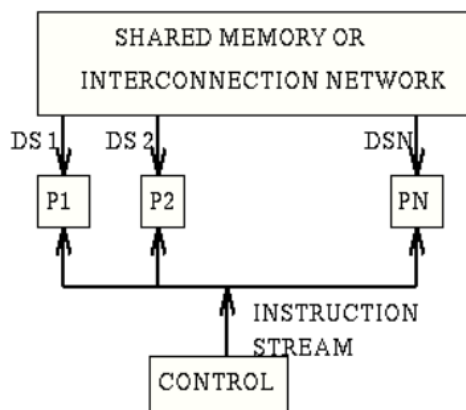
SISD (Single Instruction Single Data stream) - računari sa jednim tokom instrukcija i jednim tokom podataka.



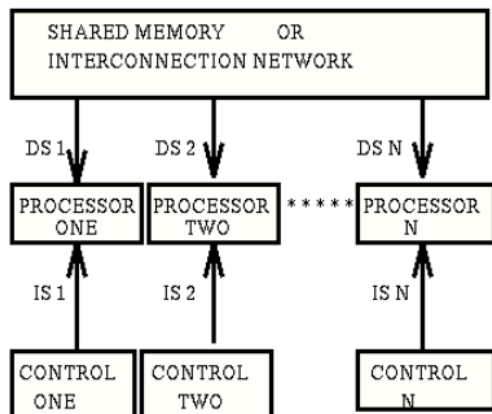
MISD (Multiple Instruction Single Data stream) - računari sa višestrukim tokom instrukcija i jednim tokom podataka.



SIMD (Single Instruction Multiple Data stream) - računari sa jednim tokom instrukcija i višestrukim tokom podataka.



MIMD (Multiple Instruction Multiple Data stream) - računari sa višestrukim tokom instrukcija i višestrukim tokom podataka.



2. (2) Program P se izvršava za 100 sec na jedno-procesorskom sistemu. Kada se program paralelizuje, 20 sec traje izvršenje dela programa koji se ne može paralelizovati, a ostali deo programa se može idealno paralelizovati.

- a. Ako se program izvršava na 16 procesorskom sistemu, koliko vremena će trajati izvršenje programa?

$$T(n) = T_s + T_p / n, T(16) = 20s + 80s / 16 = \mathbf{25s}$$

- b. Ako imamo beskonačno mnogo procesora na raspolaganju, koliko vremena će trajati izvršenje programa?

$$\alpha = T_s / T(1) = 0.2$$

$$\lim_{n \rightarrow \infty} S(n) = 1 / \alpha = 1 / 0.2 = 5$$

$$T(\infty) = T(1) / S(\infty) = 100 / 5 = \mathbf{20s}$$

3. (2) Sekvencijalna verzija programa se izvršava za 10000 sati. Izvršenje programa na višeprocessorskom sistemu sa 1000 procesora traje 20 sati.

- a. Koliko je ubrzanje a kolika efikasnost višeprocessorskog sistema?

$$\text{ubrzanje: } S(n) = T(1) / T(n) \quad S(1000) = 10000h / 20h = 500$$

$$\text{efikasnost: } E(n) = S(n) / n \quad E(1000) = 500 / 1000 = 0.5 * 100\% = 50\%$$

- b. Koliko bi trajalo izvršenje programa ako bi efikasnost 1000 procesorskog sistema bila 100%

$$E(1000) = 100\% = 1$$

$$1 = S(1000) / 1000 \Rightarrow S(n) = 1000$$

$$1000 = 10000h / T(n) \Rightarrow T(n) = 10000h / 1000 = \mathbf{10h}$$

4. Pretpostavimo da program koji se izvršava na višeprocessorskom sistemu ima na početku deo koji se izvršava sekvencijalno i zahteva 3ms vremena, i deo na kraju koji zahteva sekvencijalno izvršenje i traje 4ms. Između ova dva dela, program može biti podeljen na 5 jednakih delova koji se mogu izvršavati paralelno i svaki deo zahteva 16ms vremena. Koliko iznosi maksimalno ubrzanje po Amdahl-ovom zakonu?

$$n = 5$$

$$T(1) = 3ms + 4ms + 5 * 16ms = 87ms$$

$$T(n) = 3ms + 16ms + 4ms = 23ms$$

$$S(n) = 87\text{ms} / 23\text{ms} = 3.78$$

5. Pretpostavimo da imamo multiprocesor sa 100 procesora i želimo da postignemo ubrzanje od najmanje 60. Koliko programa se može izvršavati sekvencijalno?

① $n=100$ $S(n) \geq 60$ $\alpha = ?$

$$S(n) = \frac{T(1)}{T(n)} = \frac{T(1)}{\alpha T(1) + \frac{(1-\alpha)T(1)}{n}} = \frac{n}{\alpha n + 1 - \alpha}$$

$$60 \leq \frac{100}{100\alpha + 1 - \alpha}$$

$$6000\alpha + 60 - 60\alpha \leq 100$$

$$5940\alpha \leq 40$$

$$\alpha \leq 0.0067 \rightarrow 0.67\%$$

6. (2) Definisati ubrzanje i efikasnost. Eksperimentalno je utvrđeno da paralelna verzija izvesne aplikacije postiže ubrzanje od 13 kada se izvršava na 16-procesorskom sistemu. Pod pretpostavkom da se program sastoji iz delova koji su potpuno sekvencijalni i potpuno paralelni (sa zanemarljivim vremenom komunikacije i sinhronizacije) odrediti koliki je deo (frakcija) programa koji je paralelizovan. Koliko se maksimalno ubrzanje ovog programa može postići ako bi imali neograničeni broj procesa na raspolaganju?

Ubrzanje je odnos vremena izvršenja programa bez poboljšanja i vremena sa poboljšanjem.

Efikasnost je odnos ubrzanja i broja procesora, sa porastom broja procesora opada efikasnost.

$S(n) = 13$ $n = 16$ $L = ?$

$$S(n) = \frac{n}{1 + (n-1)L} = \frac{n}{L(n-1) + 1}$$

$$13 = \frac{16}{16L - L + 1}$$

$$208L - 13L + 13 = 16$$

$$195L = 3$$

$$L = \frac{3}{195} \Rightarrow L = 0.015$$

1.5% sekvencijalno
98.5% paralelizovano

$$\text{za } n \rightarrow \infty \lim S(n)_{n \rightarrow \infty} = 1 / \alpha = 1 / 0.015 = 66.67.$$

7. Potrebno je da odlučite koji od dva paralelna programa, A i B, ćete koristiti na 1024 procesorskom sistemu. Na žalost, da bi izmerili performanse imate na raspolaganju samo 16 procesorski sistem. Tokom merenja performansi na 16 procesorskom sistemu prikupili ste sledeće podatka za ubrzanje, S , i eksperimentalno određenu serijsku frakciju, α . Za koji program ćete se odlučiti? Obrazložiti.

Program A				
Broj procesora	2	4	8	16
S	1.88	3.12	4.5	5.8
α	0.06	0.09	0.11	0.12

Program B				
Broj procesora	2	4	8	16
S	1.79	2.9	4.3	5.7
α	0.12	0.12	0.12	0.12

Program A

Br. proc.	2	4	8	16
S	1.88	3.12	4.5	5.8
E	0.94	0.78	0.56	0.36
SE	1.77	2.43	2.52	2.09

Program B

Br. proc.	2	4	8	16
S	1.79	2.9	4.3	5.7
E	0.89	0.72	0.54	0.36
SE	1.59	2.08	2.32	2.05

Posmatrajući proizvod ubrzanja i efikasnosti program A prednjači u odnosu na program B, nezavisno od broja procesora koji ga izvršava.

8. Sistem se sastoji od 10 procesora. 98% koda se može paralelizovati. Da li je moguće na ovom sistemu postići ubrzanje 7? Ako da, koliko je minimalno procesora potrebno da bi se to postigo?

$$\begin{aligned}
S(h) &= \frac{h}{2h - 1 + 1} \\
7 &= \frac{h}{0,02h + 0,02 + 1} \\
7 &= \frac{h}{0,02h + 0,98} \\
0,14h + 6,86 &= h \\
0,14h - h &= -6,86 \\
h(0,14 - 1) &= -6,86 \\
h(-0,86) &= -6,86 \\
0,86h &= 6,86 \\
h &= 7,98 \approx \underline{8}
\end{aligned}$$

Scoreboard i Tomasulo

9. (2) Koje od sledećih zavisnosti mogu dovesti do problema kod protočnih procesora kod kojih je moguće izvršenje van redosleda, ali ne mogu nastupiti kod protočnih procesora kod koga je izvršenje po redosledu.
- write-after-write
 - read-after-write
 - write-after-read
 - read-after-read
10. (2) Procesor ima sledeće funkcionalne jedinice: jednu za sabiranje/oduzimanje u pokretnom zarezu sa latencijom od 4 clk ciklusa, jednu za množenje u FP sa latentnošću od 8 clk ciklusa i jednu za deljenje u FP sa latentnošću od 12 clk ciklusa. Za koliko klock ciklusa će se izvršiti sledeći niz instrukcija ako se koristi a) Scoreboard b) Tomasulov algoritam za planiranje izvršenja instrukcija. Odgovore dati popunjavanjem odgovarajućih tabela statusa instrukcija.
- MULTF F0, F2, F4
SUBF F8, F6, F2
DIVF F10, F0, F6
ADDF F6, F8, F0
- (Isti zadatak sa drugog blanketa razlika je samo: ADDF F6, F8, F2)

a. Scoreboard

Instrukcija	Issue	Read op	Ex	Wr
MULTF F0, F2, F4	1	2	3 - 10	11
SUBF F8, F6, F2	2	3	4 - 7	8
DIVF F10, F0, F6	3	12	13 - 24	25

ADDF F6, F8, F2	9	10	11 - 14	15
-----------------	---	----	---------	----

Instrukcija	Issue	Read op	Ex	Wr
MULTF F0, F2, F4	1	2	3 - 10	11
SUBF F8, F6, F2	2	3	4 - 7	8
DIVF F10, F0, F6	3	12	13 - 24	25
ADDF F6, F8, F0	9	12	13 - 16	17

b. Tomasulo

Instrukcija	Issue	Ex	Wr
MULTF F0, F2, F4	1	2 - 9	10
SUBF F8, F6, F2	2	3 - 6	7
DIVF F10, F0, F6	3	11 - 22	23
ADDF F6, F8, F2	8	9 - 12	13

Instrukcija	Issue	Ex	Wr
MULTF F0, F2, F4	1	2 - 9	10
SUBF F8, F6, F2	2	3 - 6	7
DIVF F10, F0, F6	3	11 - 22	23
ADDF F6, F8, F0	8	11 - 14	15

11. Za koliko ciklusa će se izvršiti sledeći niz instrukcija ako se koristi a) Scoreboard
b) Tomasulov algoritam za planiranje izvršenja instrukcija. Odgovore dati popunjavanjem odgovarajućih tabela statusa instrukcija.

SUB F0, F1, F2 (8 cycles)
DIV F3, F0, F4 (10 cycles)
ADD F4, F5, F6 (6 cycles)

a. Scoreboard

instrukcija	Issue	Read op	EX	WB
SUB F0, F1, F2	1	2	3 - 10	11

DIV F3, F0, F4	2	12	13 - 22	23
ADD F4, F5, F6	3	4	5 - 10	13

b. Tomasulo

instrukcija	Issue	EX	WB
SUB F0, F1, F2	1	2 - 9	10
DIV F3, F0, F4	2	11 - 20	21
ADD F4, F5, F6	3	4 - 9	11

12. Sledeći niz instrukcija se izvršava na procesoru koji implementira Tomasulov algoritam.

LD F0, 0(R1)
ADDD F2, F0, F4
MULD F4, F2, F6
ADDD F6, F8, F10
DADDI R1, R1, #8
LD F1, 0(R2)
MULD F1, F1, F8
ADDD F6, F3, F5
DADDI R2, R2, #8

Broj, tip funkcionalnih jedinica i trajanje exe faze za svaku je dat u sledećoj tabeli:

Tip funkcionalne jedinice	trajanje EXE faze	broj funkcionalnih jed.
Integer	1	1
FP adder	5	1
FP Multiplier	8	1
FP Divider	15	1

a. Za koliko klok ciklusa će se izvršiti ovaj niz instrukcija ako funkcionalne jedinice nisu protočno organizovane? Odgovor dati popunjavanjem sledeće tabele:

instrukcija	Issue	EX	WB	razlog zaustavljanja
LD FO, 0(R1)	1	2	3	
ADDD F2, F0, F4	2	4 - 8	9	čeka na F0
MULD F4, F2, F6	3	10 - 17	18	čeka na F2
ADDD F6, F8, F10	10	11 - 15	16	čeka na funkcionalnu jedinicu
ADDI R1, R1, #8	11	12	13	

LD F1, 0(R2)	14	15	17	čeka na funkcionalnu jedinicu
MULD F1, F1, F8	19	20 - 27	28	čeka na funkcionalnu jedinicu
ADDD F6, F3, F5	20	21 - 25	26	
ADDI R2, R2, #8	21	22	23	

- b. Da li bi se broj klok ciklusa smanjio ako bi funkcionalne jedinice bile protočno organizovane? Obrazložiti odgovor.

Da zato što bi se smanjio broj strukturnih hazarda.

13. Za upravljanje izvršenjem instrukcija Scoreboard koristi nekoliko tabela. Koje su to tabele? Ukratko objasniti značenje pojedinih polja u svakoj tabeli.

- **Status instrukcije (Instruction status):**
 - Govori u kojoj od 4 faze izvršenja se instrukcija nalazi.
- **Status funkcionalne jedinice (Functional unit status):**
 - Ukazuje na stanje FU. Za svaku FU postoji 9 polja u tabeli statusa:
 - *Busy* Govori da li je FU slobodna ili ne
 - *Op* Operacija koju obavlja FU (npr., + ili –)
 - *Fi* Odredišni registar
 - *Fj, Fk* Brojevi izvornih registara
 - *Qj, Qk* FU koja proizvodi podatke za izvorne registre Fj, Fk
 - *Rj, Rk* Flegovi koji ukazuju kada su Fj, Fk spremni (postavljaju se na yes kada su operandi dostupni, brišu se – postavljaju na no kada se pročitaju operandi)
- **Tabela statusa registra rezultata (Register result status):**
 - Ukazuje koja će FU izvršiti upis u koji registar, ako aktivna instrukcija ima registar kao odredište.
 - Ako ne postoji FU koja vrši upis u dati registar, odgovarajuće polje je prazno.

14. Preko kojih tabela Tomasulov algoritam upravlja izvršenjem instrukcija i kako one izgledaju. Objasniti značenje svakog polja u tabelama.

- **Rezervacione stanice**
 - *Busy* označava da li je RS i odgovarajuća FU slobodna.
 - *Op* operacija koja treba da se izvrši (npr. + ili -).
 - *Vj, Vk* vrednosti izvornih operanada S1 i S2 .
 - *Qj, Qk* rezervacione stanice koje koje generišu izvorne operande
- **Load i store baferi**
 - *A* - adresna informacija za load ili store. U startu sadrži neposredni operand, zatim efektivnu adresu kada se izračuna.
- **Status registra rezultata**
 - Ukazuje koja će FU izvršiti upis.

15. U kojoj od faza izvršenje kod Scoreboard i Tomasulovog algoritma se razrešavaju

- a. WAR hazardi
SC: Write result, T: Issue (preimenovanje)
- b. WAW hazardi
SC: Issue, T: Issue (preimenovanje)
- c. RAW hazardi
SC: Read operands, T: Execution

16. U kojoj fazi izvršenja se kod Tomasulovog algoritma obavlja preimenovanje registara?

Obavlja se u **Issue** fazi.

17. U kojoj fazi protočnog izvršenja instrukcije se razrešava grananje?

Ishod uslova grananja i adresa odredišta se određuju u trećoj fazi (EX), programski brojač PC se puni u četvrtoj fazi adresom odredišta grananja, dok će se u petoj fazi pribaviti instrukcija koja sledi instrukciju grananja.

18. Zaokružiti tačan odgovor. Hazardi su:

- a. svojstvo arhitekture
- b. svojstvo programa
- c. svojstva arhitekture i programa

19. Zaokružiti tačan odgovor. Zavisnosti po podacima su:

- a. svojstvo arhitekture
- b. svojstvo programa
- c. svojstva arhitekture i programa

20. U kojoj fazi protočnog izvršenja instrukcija se detektuju svi hazardi?

Svi hazardi se detektuju u **ID fazi** (u toku dekodiranja instrukcije).

21.

- a. Koje faze u izvršenju instrukcija postoje kod
 - Scoreboard tehnike
Issue, Read operands, Execution, Write result
 - Tomasulovog algoritma
Issue, Execution, Write result
- b. Koje aktivnosti se obavljaju u ISSUE fazi kod
 - Scoreboard tehnike
Ako je funkcionalna jedinica slobodna i ni jedna druga aktivna instrukcija nema isti odredišni registar (ne postoji WAW hazard), Scoreboard propušta instrukciju u funkcionalnu jedinicu i ažurira internu strukturu podataka.
Ovde se detektuju **strukturni** i **WAW** hazardi. Ako strukturni ili WAW hazard

postoji, onda se izdavanje svih instrukcija **zaustavlja** dok se hazard ne obriše.

- Tomasulovog algoritma

instrukcija se pribavlja iz reda čekanja.

- Ako je u pitanju FP operacija, instrukcija se izdaje ako postoji slobodna rezervaciona stanica i šalju se operandi, ako su u registrima.
- Ako je u pitanju load ili store instrukcija, ona se izdaje ako postoji slobodan load ili store bafer.
- Ako nema slobodne rezervacione stanice ili load/store bafera onda postoji strukturni hazard i instrukcija se zaustavlja.
- U ovom koraku se vrši i proces preimenovanja registara

22. (4) Pored svake od sledećih tvrdnji staviti tačno (T) ili netačno (N)

- Scoreboard vrši preimenovanje registara da bi izbegao WAW i WAR hazarde. **N**
- Scoreboard ne može forwardovati rezultat, Tomasulov može. **T**
- Tomasulov izbegava WAR i WAW hazarde. **T**
- Kod Scoreboard instrukcija se zaustavlja u Read Operands stepenu dok svi izvorni operandi ne postanu dostupni. **T**
- Kod Tomasulovog algoritma instrukcija čita sve operande iz registarskog fajla. **N**
- Jedina informacija koja se čuva u rezervacionoj stanici je kod operacije (operation), raspoloživost (busy) funkcionalne jedinice, i oznake FU koje će generisati izvorne operande. **N**
- Scoreboard vrši statičko planiranje izvršenja instrukcija a Tomasulov algoritam dinamičko. **N**
- Scoreboard zaustavlja izvršenje da bi razrešio WAR i WAW hazarde, a Tomasulov algoritam ne. **T**
- Scoreboard koristi rezervacione stanice da upravlja statusom funkcionalnih jedinica i registrar, a Tomasulov algoritam ne. **N**
- Scoreboard koristi distribuirano upravljanje a Tomasulov algoritam centralizovano. **N**

23. Šta je VLIW koju vrstu paralelizma eksploatiše VLIW?

VLIW je arhitektura(organizacija) procesora.

VLIW (Very Long Instruction Word) izdaje fiksni broj instrukcija koje su formatirane kao jedna velika instrukcija ili kao fiksni instrukcioni paket.

Eksploatiše paralelizam na nivou instrukcije.

24. Ukratko objasniti razlike izmedju VLIW i superskalarne organizacije procesora.

Superskalarni mogu da izdaju različit broj instrukcija po clk clickusu, dok VLIW izdaje fiksni broj instrukcija u vidu jedne velike instrukcije ili fiksnog instrukcionog paketa.

VLIW procesori su po definiciji sa statičkim planiranjem izvršenja instrukcija, dok superskalarni mogu da koriste i statičko i dinamičko planiranje.

25. Zaokružiti šta od navedenog važi za Superskalarne (SP), a šta za VLIW procesore.

Planiranje izvršenja instrukcija se obavlja

- a. statički uz pomoć kompajera (**SP, VLIW**)
- b. dinamički u fazi izvršenja programa (**SP, VLIW**)
- c. moguće su obe varijante (**SP, VLIW**)

Vektorski procesori

26. (4) Objasniti GCD test za detekciju loop-carry zavisnosti

NZD (Najveći Zajednički Delilac), tj. **GCD**(Greatest Common Divisor) je jednostavan i dovoljan test koji se koristi za detekciju zavisnosti.

Ako loop-carry zavisnost postoji tada **NZD(c, a)** mora deliti **(d – b)** bez ostatka, baš **(d – b)** jer je $j < k$.

Pri čemu su:

- j i k iterativni indeksi petlji
- a i c množe j i k pri indeksiranju
- b i d su konstante koje se dodaju na iterativne indekse.

Primer: $a * j + b = c * k + d$

27. (2) Pomoću GCD testa utvrditi da li u sledećem kodu postoji loop-carry zavisnost.

for $p = 1, P$

for $q = 1, Q$

for $r = 1, R$

for $s = 1, S$

$A(2p + 2q - r + 3s) = A(p - 6q + 5r + 3) * B(r + s - 4q)$

$GCD(2, 1, 3, 6, 5) = 1$

1 deli 3

=> loop-carry zavisnost može, ali ne mora postojati

28. Pomoću GCD testa utvrditi da li u sledećem kodu postoji loop-carry zavisnost.

for $i = 3, 50$

$x(i) = y(i - 2) * z(i)$

$y(i) = y(i) + z(i)$

endfor

Petlja mora prvo da se normalizuje.

for $i = 0, 47$

$x(i + 3) = y(i + 1) * z(i + 3)$

$y(i + 3) = y(i + 3) + z(i + 3)$

endfor

Provera se zavisnost izmedju clanova $y(i + 3)$ i $y(i + 1)$:

$NZD(1, 1) = 1$

1 deli -2

=> loop-carry zavisnost može, ali ne mora postojati

29. Pomoću NZD testa utvrditi da li u sledećoj petlji postoje loop carry zavisnosti

for i = 1, 100

$a(3i) = b(i) / 2$

$b(i) = a(2i - 1) + a(2i + 3)$

endfor

$a(3i) \wedge a(2i - 1)$ $a(3i) \wedge a(2i + 3)$

NZD(3, 2) = 1 NZD(3, 2) = 1

1 deli -1 1 deli 3

=> loop-carry zavisnost može, ali ne mora postojati

30. Pomoću GCD testa utvrditi da li u sledećem kodu postoji loop-carry zavisnost.

for i := 1, n1

for j := 1, n2

for k := 1, n3

$a(2 * i + 5 * j + 4 * k) = a(6 * i + 2 * j - 7 * k + 4) + \dots$

GCD(2, 5, 4, 6, -7) = 1

1 deli 4

=> loop-carry zavisnost može, ali ne mora postojati

31. (3) Pomoću GCD testa utvrditi da li u sledećem kodu postoji loop-carry zavisnost.

for i = 1, n1

for j = 1, n2

for k = 1, n3

$a(2i + 2j - 3 * k) = a(4i - 6j + 3)$

GCD(2, 3, 4, -6) = 1

1 deli 3

=> loop-carry zavisnost može da postoji

(varijanta 2: $a(2i + 2j) = a(4i - 6j + 3)$)

GCD(2, 4, -6) = 2

2 ne deli 3

=> loop-carry zavisnost sigurno ne postoji

32. Objasniti osnovne transformacije koje se mogu primeniti na gnezdo petlji.

Permutacija - omogućava zamenu mesta dvema petljama. Ako želimo da zamenimo mesta petljama po indeksima i i j, ta transformacija se opisuje pomoću matrice T koja se dobija kada se u jediničnoj matrici zamene mesta i-toj i j-toj vrsti.

Obrtanje - omogućava promenu redosleda izračunavanja po odredjenoj indeksnoj promenljivoj

Transformacija se opisuje jediničnom matricom u kojoj je u i - toj vrsti dijagonalni element jednak -1.

Krivljenje - ovom transformacijom se obavlja krivljenje jednog iterativnog indeksa u odnosu na drugi za faktor f. Transformacija se opisuje jediničnom matricom, pri čemu

se faktor f dodaje u preseku kolone i i dijagonale.
Primer matrice: krivljenje k u odnosu na j za faktor f .

1 0 0 i
0 1 0 j
0 f 1 k

33.

- a. U sledećem primeru identifikovati sve ZIV, SIV i MIV subscripte i označiti koji su separabilni a koji povezani.

for $i = 1, N$

for $j = 1, N$

for $k = 1, N$

$A(2i + 1, 5, j) = A(2i - 2, N, k) + B(i, j + i, k)$

ZIV: $(5, N)$ separabilan

SIV: $(2i + 1, 2i - 2)$ separabilan

MIV: (j, k) separabilan

- b. Na osnovu analize subscripta utvrditi da li postoji loop carry zavisnost u navedenom primeru, bez korišćenja vektora/pravca zavisnosti.

Na osnovu ZIV subscripta ne možemo da zaključimo da li ima zavisnosti jer ne znamo koliko je N .

Na SIV subscript primenjujemo GCD test.

$(2i + 1, 2i - 2)$

$a = 2, b = 1, c = 2, d = -2$

$NZD(a, c) = NZD(2, 2) = 2$

$d - b = -2 - 1 = -3$

2 ne deli -3

34. Da li je na sledećem gnezdu petlji dozvoljeno primeniti

for $i = 1, n$

for $j = 1, n$

$c(i, j) = c(i - 1, j + 1) + 1$

endfor $\{i, j\}$

- permutaciju petlji i i j
- obrtnje po indeksnoj promenljivoj i
- kompoziciju transformacija pod a i b

Tamo gde je moguće izvršiti transformaciju prikazati kako izgleda transformisano gnezdo petlji.

21) for $i=1, n$
 for $j=1, n$
 $c(i, j) = c(i-1, j+1) + 1$
 end for $\{i, j\}$

$i - i + 1 = 1$
 $j - j - 1 = -1$
 $\Rightarrow d = [1, -1]$

a) neproizvodljiva $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \hat{j} < 0 \Rightarrow HE$
 b) odzivane do i $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \hat{j} < 0 \Rightarrow HE$
 c) namoz. a) u b)
 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \hat{j} < 0 \Rightarrow HE$

35.

- Odrediti sve vektore zavisnosti i pravce zavisnosti za zadato gnezdo petlji.
- Za svaku od petlji odgovoriti da li se može paralelizovati/vektORIZOVATI, i ako ne može zbog kojih zavisnosti ne može
- Primeniti odgovarajuću transformaciju tako da se unutrašnja petlja može vektorizovati. Pokazati ceo postupak kako se dolazi do transformisane petlje.

```

for(i = 2; i < N - 1; i++) {
  for(j = 3; j < N; j++) {
    for(k = 4; k < N - 3; k++) {
      A[i][j][k] = A[2i - 4][j - 3][k - 4] + A[i + 1][j][k + 2] + A[i][j][k - 1];
    }
  }
}

```

a.

$i - 2i + 4, j - j + 3, k - k + 4 = d_1 [-i + 4, 3, 4]$
 $i - i - 1, j - j, k - k - 2 = d_2 [-1, 0, -2]$
 $i - i, j - j, k - k + 1 = d_3 [0, 0, 1]$

a) $D = \begin{bmatrix} -i+4 & -1 & 0 \\ 3 & 0 & 0 \\ 4 & -2 & 1 \end{bmatrix} = \begin{bmatrix} > & & \\ < & = & \\ < & & < \end{bmatrix} \begin{matrix} i \\ j \\ k \end{matrix}$

3 slučaja:
 1° $i \geq 4$ >
 2° $i = 4$ =
 3° $i < 4$ <

1° $\begin{bmatrix} > & & \\ < & = & \\ < & & < \end{bmatrix} \begin{matrix} i \\ j \\ k \end{matrix}$
 2° $\begin{bmatrix} = & & \\ < & = & \\ < & & < \end{bmatrix} \begin{matrix} i \\ j \\ k \end{matrix}$
 3° $\begin{bmatrix} < & & \\ < & = & \\ < & & < \end{bmatrix} \begin{matrix} i \\ j \\ k \end{matrix}$

b.

- slučaj $j > 4$:
 i - P(ne), V(da), WAR zavisnosti
 j - P(da), V(da) nema zavisnosti
 k - P(ne), V(ne) loop-carry zavisnost
- slučaj $j = 4$:

- i - P(ne), V(da), WAR zavisnosti
- j - P(ne), V(ne), loop-carry zavisnost
- k - P(ne), V(ne), loop-carry zavisnost

c. slučaj $j < 4$:

- i - P(ne), V(ne), loop-carry i WAR zavisnosti
- j - P(da), V(da), nema zavisnost
- k - P(ne), V(ne), loop-carry zavisnost

c.

c) $T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} -i+4 & -1 & 0 \\ 3 & 0 & 0 \\ 4 & -2 & 1 \end{bmatrix} = \begin{bmatrix} -i+4 & -1 & 0 \\ 4 & -2 & 1 \\ 3 & 0 & 0 \end{bmatrix} \Rightarrow \text{OK.}$

$\begin{bmatrix} u \\ v \\ w \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} i \\ j \\ k \end{bmatrix} = \begin{bmatrix} k \\ i \\ j \end{bmatrix} \Rightarrow \begin{matrix} u=i \\ v=k \\ w=j \end{matrix} \Rightarrow \text{нужно переименовать } u, v \text{ и } w.$

36. Po kojoj indeksnoj promenljivoj je moguće izvršiti vektorizaciju sledećeg gnezda petlji? Pokazati kako izgleda transformisano gnezdo petlji u kome se unutrašnja petlja može vektorizovati.

```
for(k = 0; k < L; ++k) {
    for(j = 0; j < M; ++j) {
        for(i = 0; i < N; i++) {
            a[i][j][k + 1] = a[i][j][k] + X1;
            b[i + 1][j][k] = b[i][j][k] + X2;
            c[i + 1][j + 1][k + 1] = c[i][j][k] + X3
        }
    }
}
```

$d_1 = [1, 0, 0]$
 $d_2 = [0, 0, 1]$
 $d_3 = [1, 1, 1]$

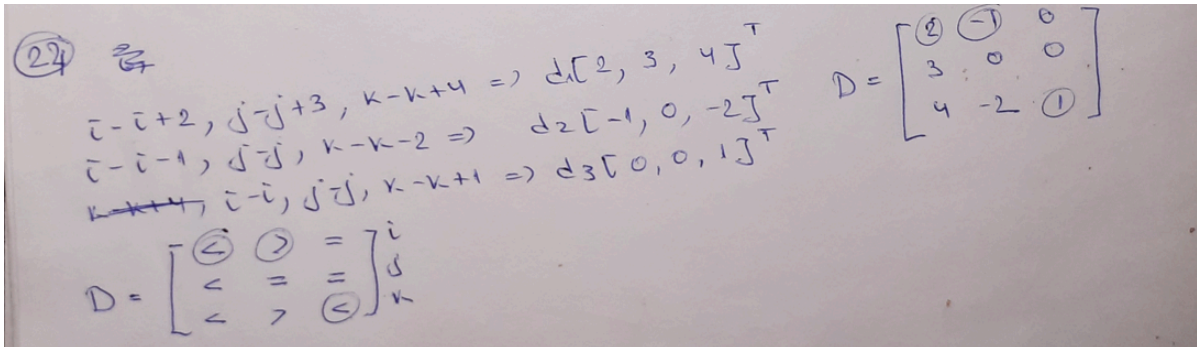
$D = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{matrix} k \\ j \\ i \end{matrix}$

$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad D \cdot T = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{array} \right] = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} > 0$

$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} k \\ j \\ i \end{bmatrix} = \begin{bmatrix} k \\ i \\ j \end{bmatrix} \Rightarrow u=k, v=i, w=j$

37. Za svaku petlju u sledećem kodu navesti da li se može paralelizovati ili ne, i u slučaju da nije navesti koje zavisnosti sprečavaju paralelizaciju (navesti tip zavisnosti i članove koji su uključeni c1, c2, c3, c4)

```
for(i = 2; i < N; i++) {
    for(j = 3; j < N; j++) {
        for(k = 4; k < N - 3; k++) {
            A[i][j][k](c1) = A[i - 2][j - 3][k - 4](c2) + A[i + 1][j][k + 2](c3) +
            A[i][j][k - 1](c4)
        }
    }
}
```



Napomena: Paralelizacija petlje je moguća na bilo kojoj dubini. Petlja ne mora da bude spoljašnja da bi se mogla paralelizovati, to je samo poželjno jer nema n-1 fork-join-a. zavisnosti: < (loop-carry), > (WAR), = (RAW koji ne sprečava ni paralelizaciju ni vektorizaciju)

petlja i paralelizacija nije moguća

zavisnosti: loop-carry(c1, c2), WAR(c1, c3)

petlja j paralelizacija je moguća

nema zavisnosti i može da se paralelizuje (zato što nije spoljna petlja pa ne nosi zavisnosti, u slučaju da je spoljna ceo red mora da bude =).

petlja k paralelizacija nije moguća

zavisnosti: loop-carry(c1, c4)

38. Zadato je sledeće gnezdo petlji

```
for i := 1 to n
  for j := 1 to n
    for k := 1 to n
      a(i, j, k) := a(i - 1, j + 1, k) + a(i, j, k - 1) + b(i, j - 1, k)
      b(i, j, k) := b(i, j - 1, k + 1) + a(i - 1, j, k)
```

- a. Da li je dozvoljeno izvršiti transformaciju sledeću transformaciju nad ovim gnezdom petlji? Obrazložiti odgovor.

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

$d > 0$

$$D = \begin{matrix} i \\ j \\ k \end{matrix} \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & -1 \end{bmatrix}$$

$$D' = T \cdot D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & -1 \end{bmatrix}$$

$$D' = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 \\ -1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

$d < 0 \Rightarrow$ nije validna transformacija.

- b. Ako je transformacija dozvoljena, kako izgleda transformisano gnezdo petlji?
ne izgleda

39. Za svako od navedenih gnezda petlji odrediti da li se spoljašnja petlja može paralelizovati bez transformacije tela petlje. Obavezno obrazložiti.

a.

```
for(i = 1; i <= n; i++) {
    for(j = 1; j <= n; j++) {
        a(i, j) = a(i, j - i) + 1;
        b(i, j) = a(i, j) + a(i, j);
    }
}
```

```
D = [0 0 0] i
     [i 0 0] j

n > 0
=> D = [= =] i
       [< =] j
```

Spoljašnja petlja se može paralelizovati zato što ne nosi zavisnosti, ni WAR ni loop-carry.

b.

```
for(i = 1; i <= n; i++) {
    for(j = 1; j <= n; j++) {
        a(i, j) = a(i - 1, j - i) + 1;
    }
}
```

Spoljašnja petlja ne može se paralelizovati zato što nosi loop-carry zavisnost.

```

b)
D = [1] i
    [i] j

n > 0
=> D = [<] i
        [<] j

```

40. Da li je u sledećem primeru moguće paralelizovati neku petlju? Da li je moguće obaviti vektorizaciju neke petlje? Obrazložiti odgovore.

```

for(i = 0; i < N; i++)
for(j = 1; j < N; j++)
A[i + 1, j - 2] = A[i, j] + A[i - 1, j - 2];

```

```

D = [1  2] i
    [-2 0] j

D = [< <] i
    [> =] j

```

Moguće je obaviti paralelizaciju i vektorizaciju unutrašnje petlje j zato što ona ne nosi zavisnosti.

41. Mnoge zavisnosti u programu mogu biti eliminisane preuređenjem koda. Zadata je sledeća petlja

```

for(i = 0; i < N; ++i)
{
    a[i] = b[i] + C;    //S1
    d[i] = a[i] + E;    //S2
}

```

Da li je korektno izvršiti preuređenje ove petlje na sledeći način (obavezno obrazložiti)

```

d[0] = a[0] + E
for(i = 1; i < N; ++i)
{
    a[i - 1] = b[i - 1] + C; //S1
    d[i] = a[i] + E;         //S2
}
a[n] = b[n] + C

```

Petlja pre preuređenja imala je samo RAW hazard kod a[i], nakon uređenja to se pretvorilo u WAR zavisnost, tako da preuređenje nije korektno.

42. (3) Ako je latentnost memorije (8, 6) clk ciklusa a vektorski korak (10, 5), koliko je najmanje memorijskih banaka potrebno da bi se elementima vektora pristupilo bez konfliktata ako broj banaka mora biti stepen dvojke?

Konflikt kod pristupa memorijskoj banci nastupa ako važi sledeći uslov:

NZS(vektorski_korak, broj_mem_banaka) / vektorski korak < latentnost_mem_sistema

$$\text{NZS}(10, n) / 10 < 8$$

$$\text{NZS}(10, n) < 80$$

n = 16

provera: $\text{NZS}(10, 16) < 80$

$$80 < 80 \Rightarrow \text{nema konflikta}$$

$$\text{NZS}(5, n) / 5 < 6$$

$$\text{NZS}(5, n) < 30$$

n = 8

provera: $\text{NZS}(5, 8) < 30$

$$40 < 30 \Rightarrow \text{nema konflikta}$$

OpenMP

43. Korišćenjem Bernštajnovih uslova utvrditi koji od procesa P1, P2 i P3, se mogu izvršavati paralelno.

P1:	$C = D \times E$	P2:	$C = L + M$	P3:	$F = B * D$
	$M = G + C$		$F = G / E$		$E = F / E$
	$A = B + C$				

$$I1 = \{D, E, G, C, B\} \quad O1 = \{C, M, A\}$$

$$I2 = \{L, M, G, E\} \quad O2 = \{C, F\}$$

$$I3 = \{B, D, F, E\} \quad O3 = \{F, E\}$$

P1 ^ P2

$$I1 \cap O2 = \{C\} \Rightarrow \text{ne može}$$

P1 ^ P3

$$I1 \cap O3 = \{E\} \Rightarrow \text{ne može}$$

P2 ^ P3

$$I2 \cap O3 = \{E\} \Rightarrow \text{ne može}$$

44. Korišćenjem Bernštajnovih uslova utvrditi koje naredbe se mogu izvršavati paralelno

P1: $C = D \times E$

P2: $M = G + C$

P3: $A = B + C$

$$\begin{aligned}
I_1 &= \{D, E\} & O_1 &= \{C\} \\
I_2 &= \{G, C\} & O_2 &= \{H\} \\
I_3 &= \{B, C\} & O_3 &= \{A\}
\end{aligned}$$

$P_1 \wedge P_2$

$$\begin{aligned}
I_1 \cap O_2 &= \{D, E\} \cap \{H\} = \emptyset \checkmark \Rightarrow \text{He moze} \\
I_2 \cap O_1 &= \{G, C\} \cap \{C\} = C \times
\end{aligned}$$

$P_1 \wedge P_3$

$$\begin{aligned}
I_1 \cap O_3 &= \{D, E\} \cap \{A\} = \emptyset \checkmark \\
I_3 \cap O_1 &= \{B, C\} \cap \{C\} = C \times \Rightarrow \text{He moze}
\end{aligned}$$

$P_2 \wedge P_3$

$$\begin{aligned}
I_2 \cap O_3 &= \{G, C\} \cap \{A\} = \emptyset \checkmark \\
I_3 \cap O_2 &= \{B, C\} \cap \{H\} = \emptyset \checkmark \Rightarrow \text{Moze} \\
O_3 \cap O_2 &= \{H\} \cap \{A\} = \emptyset \checkmark
\end{aligned}$$

45. Sledeća pitanja odnose se na OpenMP

- Na koje se sve načine može definisati broj niti koji se kreira u paralelnom regionu i koji su njihovi prioriteti?
 - if** odredba - određuje da li će parallel direktiva dovesti do kreiranja tima niti. Ako je vrednost logičkog izraza false, onda se paralelni region izvršava samo od strane jedne niti.
 - num_threads** odredba
 - omp_set_num_threads()** biblioteka funkcija - nakon početka izvršenja korišćenjem izvršne biblioteke funkcije
 - OMP_NUM_THREADS** promenljiva okruženja
 - Default** - zavisno od implementacija - ako broj niti nije eksplicitno naveden, OpenMP će automatski odrediti broj niti na osnovu dostupnih resursa sistema.
- U čemu je razlika između private i firstprivate odredbi?

Privatne promenljive paralelnog regiona nisu inicijalizovane na ulasku u paralelni region. Firstprivate promenljiva će na početku paralelnog bloka biti inicijalizovana na vrednost koju je imala istoimena promenljiva pre ulaska u paralelni region.
- U čemu je razlika između #pragma omp single i #pragma omp master direktiva?

Single direktivom se postiže da se strukturni blok izvršava od strane samo jedne niti, ali pre izvršenja nije poznato koja će nit izvršiti blok. Master direktiva označava da će samo glavna (master) nit izvršiti strukturni blok. Ova direktiva nema implicitnu barijeru na ulasku ili nakon okončanja strukturnog bloka, za razliku od direktive single.
- Koje odredbe se mogu koristiti uz direktivu #pragma omp for?

private, firstprivate, lastprivate, reduction, ordered, schedule, nowait

- e. Potrebno je paralelno izvršiti 100 iteracija petlje. Ako su kreirane 4 niti, prikazati kako će izgledati distribucija iteracija po nitima ako se koriste odredbe.

- schedule(static, 10)

1. (0-9)	...	(40-49)	...	(80-89)
2. (10-19)	...	(50-59)	...	(90-99)
3. (20-29)	...	(60-69)		
4. (30-39)	...	(70-79)		

- schedule(dynamic, 10)

1. (0-9)	...	(40-49)	...	(80-89)
2. (20-29)	...	(60-69)		
3. (10-19)	...	(50-59)	...	(90-99)
4. (30-39)	...	(70-79)		

- schedule(guided, 10)

1. (0-24)	...	(67-76)
2. (25-42)	...	(77-86)
3. (43-56)	...	(87-96)
4. (57-66)	...	(97-99)

46. U čemu je razlika između sinhronizacionih direktiva #pragma omp barrier i #pragma omp critical?

Suštinski se razlikuju - do barijere moraju da dodju sve niti istovremeno, da bi se izvršenje nastavilo. Critical označava neku kritičnu sekciju, njoj ima pristup samo jedna nit jednovremeno.

47. Koje direktive za podelu posla između niti su na raspolaganju u OpenMP?

for, sections, single, task

48. Uz OpenMP direktive mogu se koristiti određenje odredbe (klauzule). Popuniti donju tabeli navodeći oznaku x tamo gde se određena klauzula može koristiti navedenu direktivu.

	#pragma omp parallel	#pragma omp for	#pragma omp sections
shared	x		
private	x	x	x
lastprivate		x	x
firstprivate	x	x	x
nowait		x	x
schedule		x	
if	x		

num_threads	x		
ordered		x	
reduction	x	x	x

49. Sinhronizacija ili koordinacija akcija niti (thread) je nekad neophodna da bi se obezbedio korektan pristup deljivim prodacima i da bi se sprečilo narušavanje podataka. OpenMP nudi relativno mali skup jednostavih sinhronizacionih mehanizama. Koje su direktive na raspolaganju u OpenMP za sinhronizaciju između niti? Ukratko objasniti sintaksu i dejstvo svake od njih.

- **#pragma omp barrier** - Kod ovog vida sinhronizacije zahteva se da sve niti stignu do barijere pre nego što nastave dalje sa izvršenjem.
 - Sintaksa:

#pragma omp barrier
- **#pragma omp ordered** - Omogućava da se strukturni blok **u okviru paralelne petlje** izvrši sekvencijalno. Kada nit koja izvršava prvu iteraciju petlje naiđe na ordered direktivu, ona ulazi u strukturni blok bez čekanja. Da bi neka nit koja izvršava neku drugu iteraciju petlje izvršila naredbe koje se nalaze u strukturnom bloku mora da sačeka da sve niti koje izvršavaju prethodne iteracije okončaju izvršenje strukturnog bloka u okviru ordered konstrukcije. Ako se koristi ova direktiva onda se mora iskoristiti i klauzula (odredba) ordered uz paralelni region u kojem se koristi direktiva ordered.
 - Sintaksa:

#pragma omp ordered
strukturni blok
- **#pragma omp critical** - Omogućava da niti pristupaju kritičnoj sekciji uzajamno isključivo. Kada neka nit naiđe na direktivu critical ona čeka da druga nit okonča pristup kritičnoj sekciji sa datim imenom (bilo gde u programu) pre nego što ona uđe u kritičnu sekciju.
 - Sintaksa:

#pragma omp critical [(ime)]
strukturni blok
- **#pragma omp atomic** - Atomic direktiva omogućava efikasno ažuriranje deljivih promenljivih od strane više niti na hardverskoj platformi koja podržava atomične operacije. Za razliku od ostalih direktiva, ona se primenjuje samo na jednu naredbu dodeljivanja koja neposredno sledi iza konstrukcije atomic.
 - Sintaksa:

#pragma omp atomic
naredba
- **#pragma omp master** - definiše blok naredbi koji će izvršiti samo master nit. Slična je single direktivi. Ova direktiva nema implicitnu barijeru na ulasku ili nakon okončanja strukturnog bloka.

- Sintaksa:
#pragma omp master
 strukturni blok

- **bibliotečke funkcije – lock - nisu direktive**

50.

- a. Na raspolaganju je mutliprocesorski sistem sa 4 jezgra. Za paralelizaciju for petlje koja inicijalizuje na nulu gornji trougao kvadratne matrice dimenzije 100x100 koristi se OpenMP

```
#pragma openmp parallel for private(j) shchedule(...)
for(i = 0; i < 99; i++)
    for(j = i + 1; j < 100; j++)
        a[i][j] = 0.0;
```

Poređati sledeće odredbe počev od one koja će dati najbrže izvršenje do najsporijeg

- schedule(static) 6
- schedule(static, 10) 5
- schedule(static, 1) 2
- schedule(dynamic, 1) 1
- schedule(dynamic, 10) 3
- schedule(dynamic, 20) 4

- b. Koja vrednost promenljive q će biti odštampana u sledećem programu. Obrazložiti.

```
void main() {
    int q = 2;
    #pragma omp parallel num_threads(4) reduction(* : q)
    {
        q += 2;
    }
    printf("%d\n", q)
}
```

$$q = 2 * (1 + 2) * (1 + 2) * (1 + 2) * (1 + 2) = 162$$

51.

Da li se u sledećem OpenMP programu može iskoristiti nowait odredba kod prve paralelne for petlje?

Obavezno obrazložiti odgovor.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    int i, j;
    int n=1000;
    float v[n],a[n][n], b[n];
```

```

#pragma omp parallel for private(j)
for (i=0; i<n; i++) {
    for (j=0; j<i; j++) {
        v[i] +=a[i][j]*b[i];
    }
#pragma omp parallel for private(j)
for (i=1; i<n; i++) {
    for (j=i; j<n; j++) {
        v[i] = v[i-1] + a[i][j]*b[i];
    }
}
}

```

Može, prva parallel direktiva ima implicitnu barijeru koja ne može da se ukloni, pa svakako sve niti moraju da dodju do te barijere pre nego krene sledeca parallel direktiva..

52. Sledeći sekvencijalni program

```

void ccode(float a[], float b[], float c[], int n) {
    float x, y;
    int i;
    for(i = 0; i < n; i++) {
        x = a[i] - b[i];
        y = b[i] + a[i];
        c[i] = x * y;
    }
}

```

Je paralelizovan pomoću OpenMP na sledeći način

```

void ccode(float a[], float b[], float c[], int n) {
    float x, y;
    int i;
#pragma omp parallel for shared(a, b, c, n, x, y) private(i)
for(i = 0; i < n; i++) {
    x = a[i] - b[i];
    y = b[i] + a[i];
    c[i] = x * y;
}
}

```

Šta je pogrešno u ovom OpenMP programu što će uzrokovati da se ne generiše isti rezultat kao kod sekvencijalnog izvršenja? Napisati kako bi izgledala korektna verzija OpenMP programa.

```

void ccode(float a[], float b[], float c[], int n) {
    float x, y;
    int i;
#pragma omp parallel for shared(a, b, c, n) private(i) lastprivate(x, y)
for(i = 0; i < n; i++) {

```

```

        x = a[i] - b[i];
        y = b[i] + a[i];
        c[i] = x * y;
    }
}

```

53. Paralelizovati sledeći kod korišćenjem OpenMP. Broj niti treba da bude jednak broju procesora u sistemu.

```

for(j = 0; j < n; j++)
    for(i = 0; i < m; i++)
        a[i, j + 1] = a[i][j] + b[j]

D = [<] j
    [=] i

```

permutacijom dobijamo:

```

D = [=] i
    [<] j

```

može se paralelizovati po i, i da se izbegne m-1 fork-joina:

```

int t = omp_get_num_procs();
omp_set_num_threads(t);

#pragma omp parallel for
for(i = 0; i < m; i++)
    for(j = 0; j < n; j++)
        a[i, j+1] = a[i][j] + b[j]

```

54. Paralelizovati sledeću programsku sekvencu korišćenjem OpenMP

```

iper = 0;
for(i = 0; i < imax; i++) {
    iper = iper + 1;
    j[i] = iper;
}
iper = 0;
#pragma omp parallel for reduction(+:iper)
for(i = 0; i < imax; i++) {
    j[i] = i + 1;
    iper += 1;
}

ili

#pragma omp parallel for
for(i = 0; i < imax; i++) {
    j[i] = i + 1;
}

```

```
}
```

55. Paralelizovati sledeći kod korišćenjem OpenMP

```
C[0] = 1;
for(i = 1; i < N; i++) {
    C[i] = C[i - 1];
    for(j = 0; j < N; j++) {
        C[i] *= A[i, j] + B[i, j];
    }
}
C[0] = 1;
for(i = 1; i < N; i++) {
    prod = C[i - 1];
    #pragma omp parallel for reduction(* : prod)
    for(j = 0; j < N; j++) {
        prod *= A[i, j] + B[i, j];
    }
    C[i] = prod;
}
```

ili (nema N-1 fork join-a)

```
C[0] = 1;
#pragma omp parallel for ordered schedule(dynamic)
for(i = 1; i < N; i++) {
    int prod = 1;
    for(j = 0; j < N; j++) {
        prod *= A[i, j] + B[i, j];
    }
    #pragma omp ordered
    C[i] = C[i - 1] * prod;
}
```

56. Da li se sledeći programski kod na neki način može paralelizovati pomoću OpenMP

```
for(i = 0; i < 100; i++)
    a[i + 1] = a[i] + 1;
```

Ako je moguće, dati kako izgleda paralelna verzija programa. Ako nije moguće, objasniti zašto.

Transformacijom datog koda u sledeci, moze se paralelizovati:

```
#pragma omp parallel for
for(i=0; i < 100; i++)
    a[i + 1] = a[0] + i + 1;
```

57. Da li se sledeći programski kod na neki način može paralelizovati pomoću OpenMP.

```
for(i = 0; i < 100; i++)
```

```
a[i + 10] = a[i] + 1;
```

Ne. Postoji neotklonjiva loop-carry zavisnost.

D = [10] i = [<] i

```
#pragma omp parallel for
for (i = 0; i < 10; i++)
    for(j = 0; j < 10; j++)
        a[i*10 + j + 10] = a[i*10 + j] + 1;
```

58. Šta je pogrešno u sledećem OpenMP kodu (obavezno obrazložiti)

```
np = omp_get_num_threads();
#pragma omp parallel for schedule(static)
for(l = 0; l < np; l++)
    work(l);
```

omp_get_num_threads() - ova funkcija može da se pozove i izvan paralelnog regiona ali vratiće 1 zato što je aktivna samo jedna nit, master.

59. Šta je pogrešno u sledećem OpenMP kodu (obavezno obrazložiti)

```
void ccode(int n)
{
    #pragma omp parallel
    {
        #pragma omp critical
        {
            work1();
            #pragma omp barrier
            work2();
        }
    }
}
```

#pragma omp barrier ne sme biti unutar kritične sekcije. Samo jedna nit može jednovremeno ući u kritičnu sekciju, dok barijera zahteva da sve niti dođu do nje.

60. Sledeći OpenMP program se izvršava na 4-jezgarnom procesoru. Koliko se ubrzanje može postići za veliko N? Vreme potrebno za testiranje u if naredbi zanemariti.

Usvojiti da ostale operacije traju 1 vremensku jedinicu.

```
float total = 0.0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    if(i % 16 < 8) // usvojiti da testiranje zahteva 0 operacija
        out[i] = 1 * in[i]; // 1op
    else
        out[i] = 2 + in[i]; //1op
```

```

}
for(int i = 0; i < N; i++)
    total += out[i]; //1op

```

Sekvencijalno: $N + N = 2N$ operacija

Paralelno (4 niti): $N / 4 + N = 5N / 4$ operacija

Ubrzanje: $S(n) = T(1) / T(n) = 2N / (5N / 4) = 8 / 5 = 1.6$

61. Koliko je maksimalno ubrzanje moguće ostvariti izvršenjem sledećeg OpenMP programa? Objasniti, rečima, kako se program može drugačije napisati da bi se postiglo veće ubrzanje.

```

void sub_a (float a[], float b[], float c[], float d[], int n, int m)
{
    int i, j;
    #pragma omp parallel shared (a, b, c, d, n, m) private(i, j)
    {
        #pragma omp sections nowait
        {
            #pragma omp section
            for(i = 1; i < n; i++) {
                for(j = 0; j <= i; j++)
                    b[j][i] = (a[j][i] + a[j][i-1]) / 2;
            }

            #pragma omp section
            for(i = 1; i < m; i++) {
                for(j = 0; j <= i; j++)
                    d[j][i] = (c[j][i] + c[j][i - 1]) / 2;
            }
        }
    }
}

```

Maksimalno ubrzanje koje se može postići: $(n + m) / \max(n, m)$

Objašnjeno, rečima, kako se može postići veće ubrzanje: Može da se iskoristi parallel for (umesto sekcija), uočavamo da spoljašnja petlja može da se paralelizuje (pošto samo čita elemente sa indeksima $i / i - 1$, LC zavisnost po i NE postoji), pa je rešenje samo da se stavi #pragma omp parallel for na mestima gde je trenutno section.

```

void sub_a (float a[], float b[], float c[], float d[], int n, int m)
{
    int i, j;
    #pragma omp parallel shared (a, b, c, d, n, m) private(i, j)
    {

```

```

        #pragma omp for schedule(dynamic) nowait
        for(i = 1; i < n; i++) {
            for(j = 0; j <= i; j++)
                b[j][i] = (a[j][i] + a[j][i-1]) / 2;
        }

        #pragma omp for schedule(dynamic) nowait
        for(i = 1; i < m; i++) {
            for(j = 0; j <= i; j++)
                d[j][i] = (c[j][i] + c[j][i - 1]) / 2;
        }
    }
}

```

62.

- a. Zadat je sledeći OpenMP program.

```

int i, j, n = 10, t = 0;
omp_set_num_threads(2);
#pragma omp parallel private(j)
{
    #pragma omp for
    for(i = 0; i < n; i++) {
        printf("a\n");
    }
    for(j = 0; j < n; j++) {
        printf("b\n");
    }
}

```

Koliko puta će biti odštampano a? Koliko puta će biti odštampano b?

a - 10

b - 20 - nema direktive za podelu poslova, obe niti izvršavaju celu for

- b. Zadat je sledeći OpenMP program

```

int i, j, n = 100000, t = 0;
#pragma omp parallel for
    for(i = 0; i < n; i++) {
        t = t + 1;
    }
printf("t = %d\n", t);
}

```

Koja od sledećih tvrdnji je tačna, kratko obrazložiti

- a. Uvek će biti odštampano 100000

b. Odštampana vrednost će biti 100000, a moguće su i vrednosti manje od 100000

- c. Bilo koja vrednost između -MAXINT i MAXINT je moguća
Može doći do trke jer $t = t + 1$ nije atomična operacija i nije u kritičnoj sekciji.
Više niti mogu da pročitaju istu vrednost promenljive t , inkrementiraju je i upišu isti rezultat.

63. (2) Koja vrednost promenljive j će biti odštampana nakon izvršenja sledećeg koda.
Šta bi bilo odštampano ako bi umesto master bila iskorišćena single direktiva?

```
int j;  
#pragma omp threadprivate(j)  
int main() {  
    j = 1;  
    #pragma omp parallel copyin(j) {  
        #pragma omp master  
        j = 2;  
    }  
    printf("j = %d\n", j);  
}
```

Vrednost j nakon izvršenja biće 2, a u slučaju single 1 ili 2 u zavisnosti da li je ta single nit master. (kakva navlakuša)

64.

- a. Ubaciti odgovarajuće OpenMP direktive tako da se paralelizuje sledeći program

```
int i;  
double A[N] = { ... }, B[N] = {...}, C[N], D[N];  
const double c = ...;  
const double x = ...;  
double y;  
void main() {  
    #pragma omp parallel for lastprivate(y)  
    for(i = 0; i < N; i++) {  
        y = sqrt(A[i]);  
        D[i] = y + A[i] / (x * x);  
    }  
}
```

- b. Koje odredbe se mogu koristiti uz direktivu parallel u OpenMP?

if, num_threads, private, firstprivate, default, shared, copyin, reduction.

65. Prikazati moguće rezultate izvršenja sledećeg OpenMP programa ako se kreiraju dve niti

```
#pragma omp parallel  
{  
    printf("A ");  
    printf("race ");  
    printf("car ");  
}
```

```

}
printf("n");
1. A race car A race car n
2. A A race race car car n
3. A A race car race car n
4. A race A race car car n
5. A race A car race car n

```

66.

1. a) Prikazati šta je izlaz iz sledećeg OpenMP programa, ako postoje 4 niti.

```

int i;
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for (i=1; i <= 4; i++)
sum = sum + i;
printf("The sum is %lf\n", sum);

```

b) Prikazati šta je izlaz iz sledećeg OpenMP programa, ako postoje 4 niti.

```

int i;
double sum;
#pragma omp parallel for private (sum)
{
    sum = 0.0;
    for (i=1; i <= 4; i++)
    sum = sum + i;
    printf("The sum is %lf\n", sum);
}

```

c) Prikazati šta je izlaz iz sledećeg OpenMP programa, ako postoje 4 niti.

```

int i;
double sum = 0.0;
#pragma omp parallel for shared (sum)
{
    for (i=1; i <= 4; i++)
    sum = sum + i;
}
printf("The sum is %lf\n", sum);

```

d) Prikazati šta je izlaz iz sledećeg OpenMP programa, ako postoje 4 niti.

```

int i;
double sum = 0.0;
#pragma omp parallel for private (sum)
{
    for (i=1; i <= 4; i++)
    sum = sum + 1;
}
printf("The sum is %lf\n", sum);

```

- a) **The sum is 10**
- b) **Greška**
- c) **Greška**
- d) **Greška**

67. Zadat je sledeći OpenMP program:

```

void main() {
    int c1 = 0;
    int c2 = 0;
    int i;
    omp_set_num_threads(4);

```

```

#pragma omp parallel
{
    #pragma omp atomic
    c1++;
    printf("par: %d\n", omp_get_thread_num()); //L1
    #pragma omp master
    {
        #pragma omp critical
        {
            c2++;
            printf("master: %d\n", omp_get_thread_num()); //L2
        }
    }
    #pragma omp single
    {
        #pragma omp critical
        {
            printf("single: %d\n", omp_get_thread_num()); //L3
            c2++;
        }
    }
}
printf("c1:%d, c2:%d\n", c1, c2); //L4
}

```

Koje vrednosti će biti odštampane:

- u naredni označenoj sa L1
par: 0
par: 1
par: 2
par: 3 (nekim redosledom)
- u naredni označenoj sa L2
master: 0
- u naredni označenoj sa L3
single: pa neki identifikator od 4 procesa (0, 1, 2 ili 3)
- u naredni označenoj sa L4
c1: 4, c2: 2

68. Napisati program na OpenMP kojim se vrši paralelno izračunavanje proizvoda bez korišćenja odredbe reduction i sa korišćenjem ove odredbe. Predvideti štampanje rezultata.

$$P = \prod_{i=0}^{99} x_i$$

a. sa redukcijom

```
...
int prod = 1;
#pragma omp parallel for reduction(* : prod)
for(int i = 0; i < 100; i++) {
    prod *= x[i];
}
...
```

b. bez redukcije

```
...
#pragma omp parallel private(locProd) shared(x,n,prod)
{
    locProd = 1;
    #pragma omp for
    for(int i = 0; i < 100; i++)
        locProd *= x[i];

    #pragma omp atomic
    prod *= locProd;
}
...
```

69. Kako se sledeći kod može modifikovati tako da se postigne maksimalni paralelizam, a da konačni rezultat bude kao da se program sekvencijalno izvršio, tj. da na kraju x, a, b i c imaju iste vrednosti koje bi imale kada se kod sekvencijalnog izvršava. Prikazati kako bi izgledala paralelna verzija takvog koda u OpenMP.

```
x++;
a = x + 2;
b = a + 3;
c++;
```

```
#pragma omp parallel sections
{
    #pragma omp single
    {
        x++;
    }
    #pragma omp section nowait
    a = x + 2;
    #pragma omp section nowait
    b = x + 5;
    #pragma omp section nowait
    c++;
}
```

SIMD i MIMD

70. Kako se može obaviti maskiranje procesnih elemenata u SIMD?

- **Direktno maskiranje**
 - Statički određeno kompajlerom i kontrolnom jedinicom.
 - Po 1 bit za svaki procesni element.
 - Omogućava maskiranje proizvoljnog skupa.
- **Maskiranje pomoću adrese PE**
 - Za $N = 2^m$ PE, koristi se m-bitna maska.
 - Svaka pozicija u maski može imati vrednost 0, 1 ili X.
 - Aktivni su oni PE čija se adresa poklapa sa maskom.
 - Nije moguće maskirati proizvoljan podskup PE. (pr. nije moguće maskirati 000 i 111)
- **Maskiranje podacima** (dinamičko maskiranje)
 - Svi procesni elementi izvrše test uslov zatim postavljaju svoj status na 1 ili 0, i podele se na dva disjunktne skupa, oni koji će izvršiti naredbe ako je uslov zadovoljen i oni koji će izvršiti naredbe kad uslov nije zadovoljen.

71. Kojim sprežnim funkcijama su definisane sledeće sprežne mreže:

a. mešanje-zamena (shuffle-exchange)

$$S(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-2}, \dots, b_1, b_0, b_{m-1}$$

$$E(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-1}, b_{m-2}, \dots, b_1, \bar{b}_0$$

b. kub

Definisana sa $m = \log_2 N$ sprežnih f-ja

$$C_i(b_{m-1}, \dots, b_i, \dots, b_1, b_0) = b_{m-1}, \dots, \bar{b}_i, \dots, b_1, b_0,$$
$$i = 0, 1, \dots, m-1$$

c. rešetka

Rešetka (mesh): (dijametar $O(\sqrt{n})$, $r = \sqrt{N}$, N – broj PE)

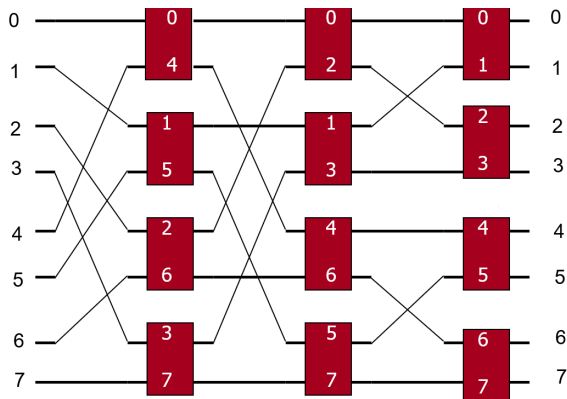
Definisana sa 4 sprežne f-je:

- $M_{+1}(x) = (x+1) \bmod N$
- $M_{-1}(x) = (x-1) \bmod N$
- $M_{+r}(x) = (x+r) \bmod N$
- $M_{-r}(x) = (x-r) \bmod N$, $r = \sqrt{N}$

72. Višestepena sprežna mreža cub.

- Ili ti generalizovani kub.
- Sastoji se od $\log_2 N = m$ stepena i $N / 2$ komutacionih elemenata po stepenu.

- Svaki stepen realizuje jednu sprežnu funkciju kub mreže.



73. U statičkoj hiperkub mreži potrebno je poslati poruku iz čvora (0, 0, 1, 0, 0, 0) u čvor (1, 0, 1, 0, 1, 0). Prikazati preko kojih čvorova će poruka biti preneti do odredišta.

001000 => 101000 => 101010

74. Kako se obavlja rutiranje poruka u sprežnoj mreži ako je ona realizovana kao

a. Statička sprežna mreža

Rutiranje u hiperkubu

- Neka je $s(i)$ adresa izvora, $d(i)$ adresa odredišta i $\sigma(i)$ adresa čvora koji je na putu od $s(i)$ do $d(i)$.
- Uporediti bitove adresa $d(i)$ sa $\sigma(i)$ s leva u desno
- Identifikovati prvu bit poziciju na kojoj se ove dve adrese razlikuju
- Proslediti paket susedu $n(i)$ tako da se adrese $n(i)$ i $\sigma(i)$ razlikuju samo na uočenoj bitskoj poziciji.

Npr. $s(i) = (1,0,1,0)$, $d(i) = (0,1,0,1)$ ide preko čvorova:

$(1,0,1,0) \rightarrow (0,0,1,0) \rightarrow (0,1,1,0) \rightarrow (0,1,0,0) \rightarrow (0,1,0,1)$

b. Jednostepena dinamička sprežna mreža

Dinamičke SM se definišu skupom sprežnih funkcija

- Sprežna f-ja S_f preslikava adresu X jednog PE u adresu $S_f(X)$ drugog PE sa kojim se ostvaruje povezivanje.
- Kada se sprežna f-ja primeni na adresu jedne komponente sistema, dobija se adresa druge komponente koja može direktno primiti podatak.
- Da bi se preneo podatak od jednog PE do drugog PE mora se izvršiti programirana sekvenca jedne ili više sprežnih f-ja.
- Kod SIMD sistema svi aktivni PE-ovi moraju izvršavati istu sprežnu f-ju u datom trenutku
- Sprežne f-je su deo SIMD programa

c. Višestepena dinamička sprežna mreža

Rutiranje u generalizovanom kubu.

Određivanje puta pomoću

EX-OR zaglavlja:

- poruci se dodaje zaglavlje dužine m bitova

- zaglavlje se dobija primenom EX-OR operacija na binarnim adresama izvora i odredišta adresa izvora $S = s_{m-1} s_{m-2} \dots s_1 s_0$
- adresa odredišta $D = d_{m-1} d_{m-2} \dots d_1 d_0$
- zaglavlje $T = SD = t_{m-1} t_{m-2} \dots t_1 t_0$

Pomoću odredišne adrese...

75. Navesti osnovne karakteristike i prednosti i mane multiprocesorskih i multiračunarskih sistema.

Multiprocesor

- Sadrži dva ili više homogenih procesora podjednakih mogućnosti.
- Svi procesori dele pristup zajedničkoj memoriji ali svaki može imati i malu lokalnu memoriju.
- Procesor je sa zajedničkom memorijom povezan preko procesor-memorija sprežne mreže.
- Komunikacija između procesora ostvaruje se preko deljive memorije.
- Razmena upravljačkih signala preko procesor-procesor sprežne mreže.

Prednosti: jednostavan mehanizam komunikacije preko read write operacija.

Mane: proširljivost(skalabilnost sistema), jer ne može neograničen broj procesora da se prikači na deljivu memoriju, takođe konflikti kod pristupa deljivoj memoriji, konzistentnost...

Multiračunar

- Skup procesora može biti heterogen.
- Svaki procesor ima svoju lokalnu memoriju i skup U/I uređaja.
- Ne postoji zajednička memorija.
- Komunikacija se ostvaruje slanjem poruka kroz sprežnu mrežu.
- Sprežne mreže su statičke.

Prednosti: svaki procesor ima odvojeni adresni prostor, laka proširljivost sistema, otpornost na greške...

Mane: kompleksnost, velika latentnost, teža komunikacija...

76. (2) Objasniti problem keš koherencije i kako se rešava.

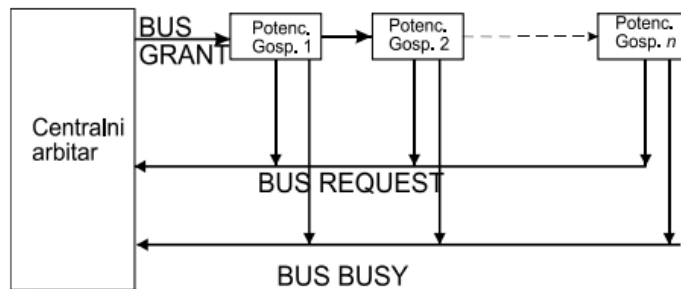
Može se desiti da u jednom trenutku više keš memorija poseduje kopiju istog podatka iz glavne memorije. Bilo koja lokalna modifikacija kopije podatka u kešu dovešće do nekonzistentnosti memorijskog sistema. To stvara nekoherenciju između keševa što znači različiti procesori imaju različite vrednosti za isti podatak.

Postoji više prilaza za rešavanje ovog problema

- Hardverski implementirani protokoli za postizanje keš koherencije
- Softverske tehnike

77. (2) Objasniti arbitražu na magistrali koja se zasniva na deljivim zahtevima i lančanom zahvatanju.

- Svaki potencijalni gospodar magistrale izdaje zahtev za dodelu magistrale preko linije BUS REQUEST. Svi zahtevi su povezani žičanom ILI logikom.
- Kada centralni arbitar primi zahtev on predaje signal BUS GRANT potencijalnom gospodaru magistrale.
 - Linija BUS GRANT povezuje sve potencijalne gospodare u lanac, tako da gospodar1 predaje signal gospodaru2, itd...
 - Gospodar koji je izdao zahtev ne prenosi signal dalje i aktivira liniju BUS BUSY čime ukazuje da je zahvatio magistralu.
 - Kada tekući gospodar završi sa magistralom on oslobađa liniju BUS BUSY i naredni ciklus arbitraže može da počne.
- Politika dodele je zasnovana na fiksnim prioritetima, gospodar koji je bliži centralnom arbitru ima viši prioritet.
- Prednost je jednostavnost i mali broj linija za povezivanje.
- Nedostatak je to što je prioritet određen fizičkom pozicijom. Može doći do izgladnjivanja gospodara sa najnižim prioritetom u slučaju da gospodar bliži arbitru često zahteva magistralu.



78. Navesti politike dodele magistrale kod multiprocesorskih sistema.

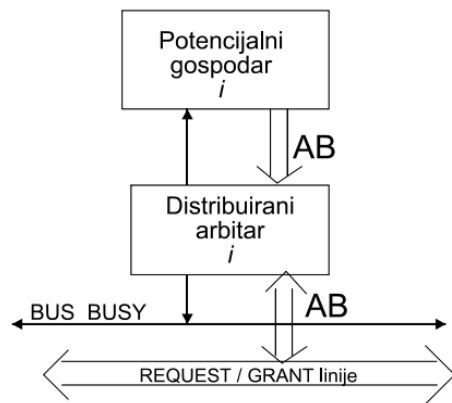
Politike mogu biti zasnovane na:

- **Prioritetu** - svakom potencijalnom gospodaru dodeljen je fiksni prioritet.
- **Nepriistrasnosti** - potencijalni gospodari imaju jednak prioritet. Svakom gospodaru koji je izdao zahtev mora se garantovati dodela magistrale pre nego što se bilo kom drugom gospodaru po drugi put dodeli magistrala.
- **Kombinovana** - politika zasnovana na prioritetu i nepriistrasnosti. U/I zahtevima se vrši dodela zasnovana na prioritetima, a procesorskim zahtevima na politici nepriistrasnosti.

79. Objasniti distribuiranu arbitražu na magistrali kod multiprocesorskih sistema?

- Hardver za arbitražu je raspodeljen po potencijalnim gospodarima magistrale.
- Svaki potencijalni gospodar ima sopstveni arbitar i jedinstveni **arbitracioni broj**.
- Kada dva ili više gospodara zahtevaju korišćenje magistrale, **pobednik je onaj koji ima viši arbitracioni broj**.
- Svaki potencijalni gospodar može poslati svoj arbitracioni broj na deljive REQUEST/GRANT linije.
- Od svih zahteva se formira **zbirni arbitracioni broj**.

- Svaki arbitar poredi svoj arbitracioni broj sa zbirnim, počev od bita najveće težine.
 - Ako je njegov broj niži to znači da je njegov prioritet niži i uklanja svoj zahtev.
 - Na kraju na linijama ostaje arbitracioni broj onog gospodara koji ima najviši prioritet i njemu se dodeljuje magistrala.



80. (2) Ako nastupi promašaj pri upisu kod obraćanja lokalnom kešu koje se akcije preduzimaju i koje je rezultujuće stanje lokalnih udaljenih kopija ako se koristi

a. Write-once protokol

Kada nastupi promašaj pri upisu procesor mora prvo da pribavi kopiju iz glavne memorije ili iz keša koji ima dirty kopiju. Ovo se postiže slanjem Read_Inv komande koja će pribaviti i ažurirati blok i invalidirati sve ostale keš kopije. Lokalna kopija se zatim nalazi u stanju dirty

b. Firefly protokol

- **nema kopije u keševima**

- pribavljanje iz glavne memorije Read_Blkl
- ažuriranje P_Write
- stanje je Dirty

- **postoji kopija u nekom kešu**

- pribavljanje iz nečijeg keša Read_Blkl
- ažuriranje P_Write
- ažuriranje svih kopija Write_Update
- stanje je Shared

Objasniti koje se lokalne procesorke komande i komande konzistencije tom prilikom emituju.

81. (3) (32, 64) PE povezana u hiperkub višestepenom sprežnom mrežom.

a. Koliko stepena ima mreža i koliko komutacionih elemenata?

$N = 32 \Rightarrow \log_2 N = 5$ stepena

$N / 2 = 16$ komutacionih elemenata **po stepenu**

$N = 64 \Rightarrow \log_2 N = 6$ stepena

$N / 2 = 32$ komutacionih elemenata **po stepenu**

- b. Iz PE-a sa adresom 21 potrebno je poslati poruku PE-ovima sa adresama 9, 13, 25, i 29. Da li je moguće obaviti emisiju? Ako jeste kako izgleda zaglavlje koje se dodaje poruci?

Da li je moguće obaviti emisiju?

- Broj odredišta je stepen dvojke? **DA**
- Odredišne adrese se razlikuju na $j = 2$ pozicija, a poklapaju na $m - j = 3$ pozicija? **DA**

=> Emisiju je moguće obaviti

21: 10101

9: 01001 $T_9 = 10101 \oplus 01001 = 11100$

13: 01101 $T_{13} = 10101 \oplus 01101 = 11000$

25: 11001 $T_{25} = 10101 \oplus 11001 = 01100$

29: 11101 $T_{29} = 10101 \oplus 11101 = 01000$

R: $T_9 = 11100$

B: $T_9 \oplus T_{29}$ ili $T_{13} \oplus T_{25}$

{R, B} = {11100, 10100}

- c. Iz PE-a sa adresom 60 potrebno je poslati poruku PE-ovima sa adresama 37, 39, 53, 55. Da li je moguće obaviti emisiju? Ako jeste kako izgleda zaglavlje koje se dodaje poruci?

Da li je moguće obaviti emisiju?

- Broj odredišta je stepen dvojke? **DA**
- Odredišne adrese se razlikuju na $j = 2$ pozicija, a poklapaju na $m - j = 4$ pozicija? **DA**

=> Emisiju je moguće obaviti

60: 111100

37: 100101 $T_{37} = 111100 \oplus 100101 = 011001$

39: 100111 $T_{39} = 111100 \oplus 100111 = 011011$

53: 110101 $T_{53} = 111100 \oplus 110101 = 001001$

55: 110111 $T_{55} = 111100 \oplus 110111 = 001011$

R: $T_{37} = 011001$

B: $T_{37} \oplus T_{55} = 010010$

{R, B} = {011001, 010010}

82.

- a. U ESC (Extra Stage Cube) višestepenoj sprežnoj mreži koja povezuje 16 PE potrebno je poslati poruku iz čvora sa adresom 5 u čvor sa adresom 11. Kako izgleda zaglavlje poruke za primarni i sekundarni put?

$S = 0101, D = 1011 \Rightarrow T = S \oplus D = 1110$

Primarni put: $0t(m-1)t(m-2)...t_0 = 01110$

Sekundarni put: $1t(m-1)t(m-2)...t_0 = 11111$

- b. U ESC (Extra Stage Cube) višestepenoj sprežnoj mreži koja povezuje 16 PE potrebno je poslati poruku iz čvora sa adresom 3 u čvorove sa adresama 4 i 6 (obaviti emisiju). Kako izgleda zaglavlje poruke za primarni i sekundarni put?

$$N = 16 = 2^m \Rightarrow m = 4$$

Emisija prema 2^j elementa, $j = 1 \Rightarrow$ Adrese moraju da se razlikuju na najviše j bit-pozicija, i iste treba da budu na $m-j$ bit pozicija:

$$S = 0011$$

$$E = 0100$$

$$F = 0110$$

$$TE = S \oplus E = 0111$$

$$TF = S \oplus F = 0101$$

Zaglavlje je kod emisije određeno skupom $\{R, B\}$

$$R = TE \text{ (može i } TF) = 0111$$

$$B = TE \oplus TF = E \oplus F = 0010$$

$$\text{Primarni put: } R_{\text{novi}} = 0r_{(m-1)}r_{(m-2)}\dots r_0 = 00111$$

$$\text{Sekundarni put: } R_{\text{novi}} = 1r_{(m-2)}r_{(m-2)}\dots \sim r_0 = 10110$$

$$B = 0 + 0010 = 00010$$

- c. U hiperkub statičkoj sprežnoj mreži sa 16 procesora potrebo je poslati poruku iz čvora sa adresom 1001 u čvor 0110. Preko kojih čvorova će poruka biti prosleđena do odredišnog čvora? Prikazati kako se došlo do rešenja.

$$S = 1001 \rightarrow 0001 \rightarrow 0101 \rightarrow 0111 \rightarrow 0110 = D$$

83. Potrebno je povezati 625 procesora. Na raspolaganju su vam sledeće sprežne mreže: a) magistrala, b) potpuno povezana mreža i c) 25x25 rešetka (mesh). Za svaku od sprežnih mreža navesti po jednu prednost i nedostatak. Koju sprežnu mrežu biste odabrali. Obrazložiti.

a) **Magistrala**

Prednost: najmanja cena

Mana: najmanja propusnost (lako dolazi do zagušenja i nije pouzdana)

b) **Potpuno povezana**

Prednost: rutiranje

Mana: najveća cena, veliki broj komutacionih elemenata

c) **Mesh 25x25**

Prednost: manje komutacionih elemenata od crossbar mreže, a brža je od magistrale

Mana: rutiranje

Biramo **Mesh** pokémona - (obrazloženje) - dimenzije rešetke odgovaraju broju procesora, brži je prenos od magistrale i nema previše komutacionih elemenata.

84. (2) Multiprocesorski sistem se sastoji od tri procesora P1, P2, P3. Deljiva memorija se sastoji od 4 bloka x, y, z i w. Svaki procesor ima keš u kojime se može naći samo jedan blok u datom trenutku. Za postizanje keš koherencije koristi se Write-once

protokol. Pretpostaviti da su keš memorije inicijalno prazne a da su sadržaji memorijskih blokova
x 10, y 20, z 30, w 40
(verzija 2: x 10, y 30, z 80, w 20)
Prikazati sadržaje keševa i glavne memorije i stanje keš blokova i GM nakon svake operacije.

akcija	keš P1	keš P2	keš P3	glavna memorija x y z w
P1 read(x)	10 (Valid)			x: 10, y: 20, z: 30, w: 40
P2 read(x)	10 (Valid)	10 (Valid)		x: 10, y: 20, z: 30, w: 40
P3 read(x)	10 (Valid)	10 (Valid)	10 (Valid)	x: 10, y: 20, z: 30, w: 40
P1 x += 25	35 (Reserved)	10 (Invalid)	10 (Invalid)	x: 35, y: 20, z: 30, w: 40
P1 read(z)	30 (Valid)	10 (Invalid)	10 (Invalid)	x: 35, y: 20, z: 30, w: 40
P2 read(x)	30 (Valid)	35 (Valid)	10 (Invalid)	x: 35, y: 20, z: 30, w: 40
P3 x = 15	30 (Valid)	35 (Invalid)	15 (Dirty)	x: 35, y: 20, z: 30, w: 40
P1 z += 1	31 (Reserved)	35 (Invalid)	15 (Dirty)	x: 35, y: 20, z: 31, w: 40

85. U dvo-procesorskom sistemu koristi se write once protokol za postizanje keš koherencije. Incijalno su keš memorije procesora prazne. Promenljiva X koja se nalazi u glavnoj memoriji ima vrednost 0.

Aktivnost procesora	Događaj u P1	Komanda konzistencije	Sadržaj keša u P1	Događaj u P2	Komanda konzistencije	Sadržaj keša u P2	Sadržaj memorijske lokacije X
P1 čita X	promašaj pri čitanju	Read_Bl	0 (Valid)				0
P2 čita X			0 (Valid)	promašaj pri čitanju	Read_Bl	0 (Valid)	0
P1 inkrementira X	pogodak pri upisu	Write_inv	1 (Reserved)			0 (Invalid)	1
P2 inkrementira X			1(Invalid)	promašaj pri upisu	Read_Inv	2 (dirty)	1

86. Sledeći program izvršava se paralelno na dvo procesorskom sistemu
I1: LOAD R1, [A] // čita sadržaj memorijske lokacije na adresi A i pamti u registar R1.

I2: LOAD R2, [B] // čita sadržaj memorijske lokacije na adresi B i pamti u registar R2.

I3: ADD R3, R1, R2 //sabira sadržaje registrar R1 i R2 i pamti registar R3.

I4: STORE [A], R3 //pamti sadržaj registra R3 u memoriju na adresu A.

I5: SUB R4, R4, 1 //oduzima 1 od R4.

I6: BNZ R4, I1 //grananje na I1 ako R4 nije 0.

Ako procesori P1 i P2 izvršavaju instrukcije u sledećem redosledu:

I ako se promenljive A i B nalaze u dva različita keš bloka, popuniti sledeću tabelu ako se koristi write-once protokol za postizanje keš koherencije. Inicijalno su keševi prazni.

Instrukcija	Stanje u kešu P1 za blok A	Stanje u kešu P1 za blok B	Stanje u kešu P2 za blok A	Stanje u kešu P2 za blok B
P1: I1	Valid			
P2: I1	Valid		Valid	
P1: I2	Valid	Valid	Valid	
P1: I3	Valid	Valid	Valid	
P1: I4	Reserved	Valid	Invalid	
P2: I2	Reserved	Valid	Invalid	Valid
P2: I3	Reserved	Valid	Invalid	Valid
P2: I4	Invalid	Valid	Dirty	Valid

87. Prikazati sadržaje keš memorija i glavne memorije i stanje keš blokova nakon svake operacije navedene u tabeli za slučaj da se koristi write-once protokol.

		Glavna memorija	P keš		Q keš	
korak	dogadjaj	lokacija X	lokacija X	stanje bloka	lokacija X	stanje bloka
0	Početna vrednost	5				
1	P čita X	5	5	Valid		
2	Q čita X	5	5	Valid	5	Valid
3	Q X += 5	10	5	Invalid	10	Reserved
4	Q čita X	10	5	Invalid	10	Reserved
5	Q X += 5	10	5	Invalid	15	Dirty
6	P X += 5	10	20	Dirty	15	Invalid

7	Q čita X	20	20	Valid	20	Valid
---	----------	----	----	-------	----	-------

88. Multiprocesorski sistem sa 4 procesora koristi Write-once protokol za održavanje keš koherencije. Inicijalno su keš memorije procesora prazne. Pretpostavimo da se lokacije X i Y nalaze u istom keš bloku. U sledećoj tabeli staviti oznaku * u prvu kolonu svake vrste ako se zahteva pribavljanje kopije keš bloka iz nekog drugog keša ili glavne memorije.

potrebno pribaviti blok	CPU 0	CPU 1	CPU 2	CPU 3
*		Store X D		
		Load X D		
*		I	Store Y D	
		I	Load X D	
*		I	V	Load X V
*	Load X V	I	V	V
*	V	Load X V	V	V
	I	I	Store Y R	I
*	I	Load X V	I	I
	I	Load Y V	I	I
*	I	V	I	Load X V
	I	I	I	Store Y R

89. Pretpostavimo da imamo multiprocesorski sistem sa (1024, 512) procesora. Svaki procesor ima lokalnu keš memoriju kapaciteta 1 MB. Kapacitet glavne memorije je 1GB. Veličina keš bloka je 64B.

- a. Ako se koristi write-once protokol za postizanje keš koherencije koliko bitova stanja zahteva implementacija ovog protokola.

Keš blok ima 4 stanja: Valid, Invalid, Reserved i Dirty, koje kodiramo sa 2 bita.

Broj keš blokova u lokalnoj keš memoriji procesora: $1\text{MB} / 64\text{B} = 2^{20} / 2^6 = 2^{14}$

Ukupan broj bitova stanja dobijamo kada pomnožimo broj procesora sa brojem keš blokova po procesoru i sve to sa 2b kojim kodiramo stanje keš bloka

$$2b * 1024 * 2^{14} = 2b * 2^{10} * 2^{14} = 2^{25} b$$

- b. Ako se koristi direktorijumska šema sa potpuno preslikanim adresarima koliko bitova stanja je potrebno za implementaciju protokola.

Jedan dirty bit kao i po 1 bit za svaki keš blok

Glavna memorija: $(1024 + 1) * 1\text{GB} / 64\text{B}$

Lokalni keševi: $1024 * 2\text{b} * 1\text{MB} / 64\text{B}$

ukupno = $(1024 + 1) * 1\text{GB} / 64\text{B} + 1024 * 2\text{b} * 1\text{MB} / 64\text{B}$

- c. Koju od ove dve šeme biste iskoristili za postizanje keš koherencije? Obrazložiti.
Direktorijumske šeme

90. Multiprocesorski sistem od 256 procesora koristi direktorijumsku šemu sa ulančanim adresarima za održavanje keš koherencije. Svaki procesor ima privatnu keš memoriju kapaciteta 1MB, a veličina keš bloka je 64 byte. Veličina glavne memorije je 4 GB. Koliko ukupno bitova stanja zahteva implementacija ovog protokola?

U direktorijumu jednog memorijskog bloka sa ulančanim adresarima, jedan valid bit i m bitova za adresu prvog procesora u lancu, gde je $N = 2^m$, u ovom slučaju $N = 256 \Rightarrow m = 8$.

Kod keš bloka u svakom procesoru se koristi m bitova za adresu sledećeg ulančanog procesora.

S obzirom da je veličina glavne memorije 4GB a veličina bloka 64B, to znači da u memoriji ima $2^{32} / 2^6 = 2^{26}$ memorijskih blokova.

U svakom procesoru ima $1\text{MB} / 64\text{B} = 2^{20} / 2^6 = 2^{14}$ keš bloka.

Što znači da ima $N * 2^{14} = 2^8 * 2^{14} = 2^{22}$ keš blokova u sistemu.

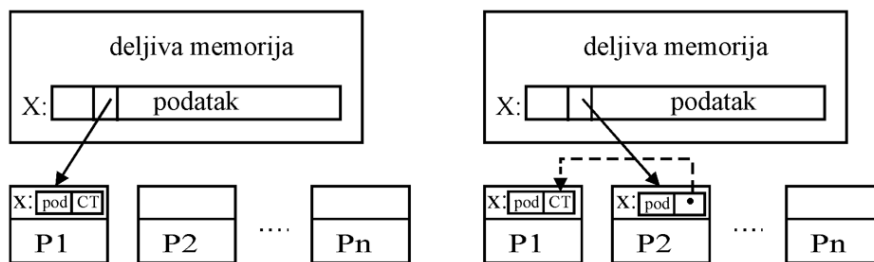
Ukupno bitova je:

$N_b = (1 + m) * \text{br_mem_blokova} + m * \text{br_keš_blokova}$

$N_b = 9 * 2^{26} + 8 * 2^{22} = 9 * 2^{26} + 2^{25}$

91. (2) Objasniti kako izgleda formiranje direktorijumskog lanca ako se koristi direktorijumska šema sa ulančanim adresarima.

- Pretpostavimo da u sistemu ne postoji kopija bloka sa lokacijom X.
- Ako P1 čita lokaciju X, glavna memorija šalje kopiju bloka kešu procesora P1 i ujedno pokazivač na kraj lanca CT (Chain Termination).
- U adresaru glavne memorije čuva se pokazivač na procesor P1
- Ako P2 zahteva čitanje lokacije X, glavna memorija šalje kopiju bloka kešu procesora P2 i pokazivač na P1.
- Sada se u adresaru glavne memorije čuva pokazivač na procesor P2
- Ponavljanjem ovih koraka svi keševi mogu dobiti kopiju bloka.
- Neka jedan od procesora, recimo P3, zahteva da modifikuje sadržaj lokacije X.
- Da bi se održala konzistentnost potrebno je da se komanda invalidacije prosledi kroz lanac.
- Kontroler glavne memorije ne daje dozvolu modifikacije procesoru P3 sve dok ne primi potvrdu o invalidaciji od procesora koji sadrži CT pointer.



92. Dva 1024 multicore procesora koristi direktorijumske šeme za održavanje keš koherencije. Jedan koristi varijantu sa potpuno preslikanim adresarima, a drugi sa ograničenim adresarima pri čemu je maksimalni broj jednovremenih kopija 8.

- a. Koliko invalidacija će obaviti sistem koji koristi potpuno preslikane adresare a koliko sistem sa ograničenim adresarima nakon što svaki od 1024 procesora pročita istu deljivu promenljivu?

protokol	broj invalidacija
full-map directory	0
limited directory (8 max)	1016

- b. Koliko invalidacija će obaviti sistem koji koristi potpuno preslikane adresare a koliko sistem sa ograničenim adresarima nakon što svaki od 1024 procesora pročita istu deljivu promenljivu a zatim jedan procesor modifikuje tu promenljivu?

protokol	broj invalidacija
full-map directory	1023
limited directory (8 max)	7 + 1016

93. Objasniti kako se održava konzistentnost podataka kod direktorijumskih šema

Svaki memorijski modul sadrži poseban direktorijum koji beleži stanje i prisustvo memorijskog bloka u određenom kešu.

Svaki direktorijumski ulaz u glavnoj memoriji sadrži po jedan bit za svaki procesor u sistemu i jedan "dirty" bit.

Svaki bit pridružen procesoru ukazuje na status bloka u odgovarajućem procesorskom kešu (prisutan ili ne).

ako je "dirty" bit postavljen, tada je jedan i samo jedan procesorski bit postavljen i taj procesor ima pravo da vrši upis u kešovani blok (tj. da ga modifikuje).

Svaki keš direktorijum sadrži dva bita stanja za svaki kešovani blok.

- Jedan bit ukazuje da li je blok važeći, a
- drugi da li se u važeći blok može vršiti upis.

94.

- a. U čemu je razlika između čvrsto spregnutih i slabo spregnutih MIMD sistema?

Multiprocesor sastoji se od homogenih procesora, multiračunar može od heterogenih.

Multiprocesor imaju zajedničku memoriju i skup U/I uređaja, multiračunar nema, svaki procesor ima svoju lokalnu.

Kod multiprocesora komunikacija se obavlja putem deljive memorije, dok kod multiračunara slanjem poruka kroz sprežnu mrežu.

...

- b. Pretpostavimo da projektujete slabo spregnuti MIMD sistem. Koju vrstu keš koherentih protokola biste koristili? Zašto?

Slabo spregnuti MIMD sistem nema deljivu memoriju.

- c. U multiprocesorskom sistemu sa privatnim keš memorijama deljivi podatak se čita od strane svih procesora u svakom klok ciklusu a modifikuje jednom na svakih 1000 ciklusa. Koju vrstu keš koherentog protokola biste koristili? Zašto?

Firefly.

- d. Pretpostavimo da se koristi keš koherentni protkol baziran na direktorijumskim šemama. Pretpostavimo da za neki blok A, direktorijumski ulaz u glavnoj memoriji ima sve nule. Šta ovo govori o bloku?

Da ne postoji nijedna kopija tog bloka.

95. (2) Multiprocesorski sistem se sastoji od 3 procesora. Svaki procesor ima svoju keš memoriju. Prikazati kako se vrši promena bitova stanja u keš memorijama procesora i u direktorijumu glavne memorije ako se za održavanje keš koherencije koriste ograničeni adresari a maksimalni broj jednovremenih kopija je 2. Procesori pristupaju deljivoj promenljivoj X. Akcije procesora navedene su u prvoj koloni tabele. Popuniti ostale ćelije tabele.

Akcija procesora	Direktorijum u GM	Bitovi stanja u P1	Bitovi stanja u P2	Bitovi stanja u P3
P1 čita X				
P3 čita X				
P3 upisuje u X				
P1 čita X				
P2 čita X				

Neko uporno prepravlja tačno rešenje ovog zadatka...

Akcija procesora	Direktorijum u GM			Bitovi stanja u P1		Bitovi stanja u P2		Bitovi stanja u P3	
	Dirty	kopija1	kopija2	Valid	Write	Valid	Write	Valid	Write
P1 čita X	0	P1		1	0				
P3 čita X	0	P1	P3	1	0			1	0
P3 upisuje u X	1	P3		0	0			1	1
P1 čita X	0	P1	P3	1	0			1	0
P2 čita X	0	P1	P2	1	0	1	0	0	0

CUDA Pitanja

1. Neka je dat CUDA kernel i odgovarajuća funkcija koja ga poziva.

```
__global__ void vecAddKernel(float* A_d, float* B_d, float* C_d, int n)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if(i < n)
        C_d[i] = A_d[i] + B_d[i];
}

int vectAdd(float* A, float* B, float* C, int n) //podrazumevati da su nizovi A, B i C dužine n
{
    int size = n * sizeof(float);
    cudaMalloc((void **) &A_d, size);
    cudaMalloc((void **) &B_d, size);
    cudaMalloc((void **) &C_d, size);
    cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);
    vecAddKernel<<<ceil(n/256), 256>>>(A_d, B_d, C_d, n);
    cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
}
```

- Ako su A, B, C nizovi od 1000 elemenata, koliko će blokova niti biti pokrenuto?
 $\text{ceil}(1000 / 256) = 4$ bloka
- Ako su A, B, C nizovi od 1000 elemenata, koliko će warp-ova biti u svakom od blokova?
Pod pretpostavkom da jedan warp čine 32 niti, biće $256 / 32 = 8$ warpa.
- Ako su A, B, C nizovi od 1000 elemenata, koliko će niti biti u gridu?
 $4 * 256 = 1024$ niti
- Ako su A, B, C nizovi od 1000 elemenata, da li će postojati divergentnost u izvršenju kernela? Ako da, u kojoj liniji. Objasniti zašto.
Postojaće divergencija kod `if (i < n)` jer će samo niti sa vrednošću `i` manjom od `n` vršiti izračunavanja, dok će ostale preskočiti ovaj blok (pokreće se više niti od onog što je potrebno jer 1000 nije deljivo sa 256, pa se pokreće ceo nov blok sa 256 niti od kojih se više od pola ne koristi).
- Ako su A, B, C, nizovi od 768 elemenata, da li će postojati divergentnost u izvršavanju kernela? Ako da, u kojoj liniji? Objasniti zašto?
U ovom slučaju neće postojati divergencija jer će sve niti biti upošljene i izvršavaće identičan kod (768 je deljivo sa 256, ne pokreće se više niti od onog što je potrebno).
- Šta dati kernel radi?
Vrši sabiranje dva vektora, A i B, i upisuje rezultat u vektor C. Svi vektori su na globalnoj memoriji uređaja.
- Modifikovati kernel tako da svaki 256. element rezultujućeg niza sadrži srednju vrednost prethodnih 255 elemenata

```
__global__ void vecAddKernel(float* A_d, float* B_d, float* C_d, int n)
```

```

{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float C_loc;
    __shared__ float avg = 0;

    if (i < n)
    {
        if (i == 0 || i % 256 != 0)
        {
            C_loc = A_d[i] + B_d[i];
            C_d[i] = C_loc;
            atomicAdd(&avg, C_loc / 255.f);
        }

        __syncthreads();

        if (i > 0 && i % 256 == 0)
            C_d[i] = avg;
    }
}

```

2.

- a. Koji sve tipovi memorije postoje na GPU?

Postoje registri i keš memorija, globalna memorija uređaja i deljena memorija unutar Streaming Multiprocessora.

- b. Koja je razlika između shared memorije i registara na GPU?

Registri su brži od deljive memorije. Svaka nit može da pristupa svojim lokalnim promenljivama u registrima. Deljenu memoriju dele niti unutar bloka.

- c. Gde se specificira broj blokova i niti koje će izvršavati zadati kernel u CUDA programu? Specificira se prilikom poziva kernela, uz pomoć odgovarajuće izvršne konfiguracije (`myKernel<< n, m>>(args...)`), gde je `n` organizacija blokova na nivou grida, a `m` organizacija niti na nivou bloka).

- d. Da li postoje ograničenja za tip povratne vrednosti kernel funkcije u CUDA programu?

Da, `__global__` funkcije ne mogu imati povratnu vrednost (void su). One dalje mogu da pozovu `__device__` funkcije koje nemaju ovo ograničenje.

3.

- a. Zašto se u CUDA kernelima izračunavanja dele dva puta, jednom na nivou rešetke, drugi put na nivou bloka?

Zbog strukture GPU-a. On se sastoji od mnoštva Streaming Multiprocessora koji mogu da izvršavaju blokove niti.

- b. Zašto niti iz različitih blokova ne mogu da koriste istu deljivu memoriju?
Različiti blokovi se u većini slučajeva izvršavaju na različitim Streaming Multiprocessorima koji ne dele istu deljivu memoriju. Čak i kada se oni izvršavaju na istom SM-u, oni ne mogu da dele shared memoriju. Razlog je to što se unapred ne može garantovati redosled izvršavanja blokova niti, pa ne bi imalo smisla koristiti deljenu memoriju.
- c. Pretpostaviti da je neki CUDA kernel pokrenut sa 1000 blokova niti, svaki sa 256 niti. Ako je neka promenljiva deklarirana kao lokalna promenljiva u kernel funkciji, koliko verzija te promenljive će biti kreirano za vreme izvršenja kernela?
256 000
- d. Potrebno je napisati CUDA kernel za sabiranje dva vektora. Ako želimo da svaka pojedinačna nit računa jedan element rezultujućeg vektora, koji bi izraz bio najpogodniji za mapiranje indeksa niti i bloka na indeks rezultujućeg vektora:
 - i. $i = \text{threadIdx.x} + \text{threadIdx.y};$
 - ii. $i = \text{blockIdx.x} + \text{threadIdx.x};$
 - iii. $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x};$ - sve niti unutar jednog warp-a će adresirati susedne memorijske lokacije, biće više keš pogodaka pošto niti u warpu dele keš memoriju
 - iv. $i = \text{threadIdx.x} * \text{blockDim.x} + \text{blockIdx.x};$
 - v. $i = \text{blockIdx.x} * \text{threadIdx.x};$
- e. Za prethodno navedeni kernel, neka je dužina vektora 5000, neka svaka nit računa samo jedan element rezultujućeg vektora, i neka je veličina bloka 512 niti. Koliko će ukupno niti biti u gridu?
5120 (mora postojati najmanje 10 blokova niti)

4.

- a. Da li niti koji izvršavaju CUDA kernel mogu međusobno da komuniciraju? Kako? Da li postoje ograničenja?
Niti unutar istog bloka mogu da komuniciraju koristeći sinhronizaciju, atomske operacije i deljenu memoriju. Niti iz različitih blokova ne mogu da sarađuju.
- b. Da li blokovi niti koji izvršavaju isti CUDA kernel mogu međusobno da komuniciraju? Kako? Da li postoje ograničenja?
Ne.

5.

- a. Kojim memorijama mogu da pristupe sve niti u jednom bloku?
Sve niti u jednom bloku na GPU mogu da pristupe globalnog memoriji uređaja i deljenoj memoriji SM koji izvršava taj blok.
- b. Koja je prednost deljive memorije u CUDA-i?
Prednost deljive memorije u odnosu na globalnu je drastično manje vreme pristupa, pa treba težiti smanjenju pristupa globalnoj memoriji korišćenjem deljene.
- c. Šta je CUDA kernel?
CUDA kernel je program koji se izvršava na (nvidia) GPU.

- d. Napisati C/C++ funkciju *float operatorX(int a, int b, float c)* koja vraća vrednost $a * c - b$

```
float operatorX(int a, int b, float c)
{
    return a * c - b;
}
```

- e. Napisati C/C++ funkciju *float* calculateVector(float* A, int dim, int c)* koja najpre popunjava kvadratnu matricu A tako da je $A[i][j] = \text{operatorX}(i, j, c)$, a vraća vektor u kome je i-ti element dobijen kao zbir elemenata i-te vrste matrice A.

```
float* calculateVector(float* A, int dim, int c)
{
    float* vec = (float*) malloc(dim * sizeof(float));

    for (int i = 0; i < dim, i++)
    {
        float sum = 0;
        for (int j = 0; j < dim, j++)
        {
            A[i][j] = operatorX(i, j, c);
            sum += A[i][j];
        }
        vec[i] = sum;
    }
    return vec;
}
```

- f. Korišćenjem CUDA tehnologije, u programskom jeziku C/C++ napisati kernel funkciju koja odgovara funkciji *calculateVector*. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za matrice proizvoljne veličine.

**** Rešenje za neki poen ****

```
__device__ float operatorX(int a, int b, float c)
{
    return a * c - b;
}

__global__ void kernel(float* dev_a, float* out_vec, int* c) {
    int i = blockIdx.x;
    int j = threadIdx.x;

    if(j >= N) return;

    __shared__ float row_sum;
```

```

float e1 = operatorX(i, j, (float) *c);
dev_a[i * N + j] = e1;

atomicAdd(&row_sum, e1);

if(j == N - 1){
    out_vec[i] = row_sum;
}
}

```

- g. Za prethodno napisani kernel, napisati i C/C++ host kod kojim se omogućava njegovo pozivanje. Pretpostaviti veličinu bloka od 256 niti i broj blokova ne veći od 256.

```

__host__ void init(float* a, float * vec, int c) {
    float *dev_a, *dev_vec;
    int *dev_c;

    size_t a_size = N * N * sizeof(float);
    size_t vec_size = N * sizeof(float);

    cudaMalloc(&dev_a, a_size);
    cudaMalloc(&dev_vec, vec_size);
    cudaMalloc(&dev_c, sizeof(int));

    cudaMemcpy(dev_a, a, a_size, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_c, &c, sizeof(int), cudaMemcpyHostToDevice);

    kernel<<<N, 256>>>(dev_a, dev_vec, dev_c);

    cudaMemcpy(a, dev_a, a_size, cudaMemcpyDeviceToHost);
    cudaMemcpy(vec, dev_vec, vec_size, cudaMemcpyDeviceToHost);

    cudaFree(dev_a);
    cudaFree(dev_vec);
    cudaFree(dev_c);
}

```

6.

- i) Zaokružiti šta sve čini overhead kada se izvršava program na GPU:
 - a. keš promašaji na host procesoru
 - b. prenos podataka od hosta ka GPU

- c. prenos podataka od GPU ka hostu
- d. divergencija niti

Treba zaokružiti pod b, c i d.

ii) Da li su tačne ili netačne sledeće tvrdnje:

- Različite niti u bloku mogu komunicirati preko deljive memorije
- Niti u različitim blokovima mogu komunicirati preko deljive memorije

Prvo je tačno, drugo je netačno.

7.

a. Šta označavaju kvalifikatori `__global__`, `__host__`, i `__device__` u CUDA programu?

U kontekstu funkcija:

- i. `__global__` označava kernel (poziva se na hostu, izvršava na device)
- ii. `__host__` označava funkciju koja se poziva i izvršava na hostu
- iii. `__device__` označava funkciju koja se poziva i izvršava na device-u

Takođe, kvalifikator `__device__` može da označava globalnu promenljivu na GPU.

b. Da li sve niti koje se izvršavaju na GPU imaju pristup istoj deljivoj memoriji?
Zašto?

Ne, niti unutar jednog bloka mogu da pristupe samo deljivoj memoriji Streaming Multiprocessora na kom se taj blok izvršava.

c. Koji je tip povratne vrednosti kernel funkcije u CUDA programu?

`void`

трећа сепћа

